



PONTIFÍCIA UNIVERSIDADE CATÓLICA DO RIO GRANDE DO SUL
ESCOLA POLITÉCNICA

MALTA

Machine Learning Theory
and Applications Lab

Aprendizado Profundo II

BERT

Prof. Me. Otávio Parraga

Roteiro

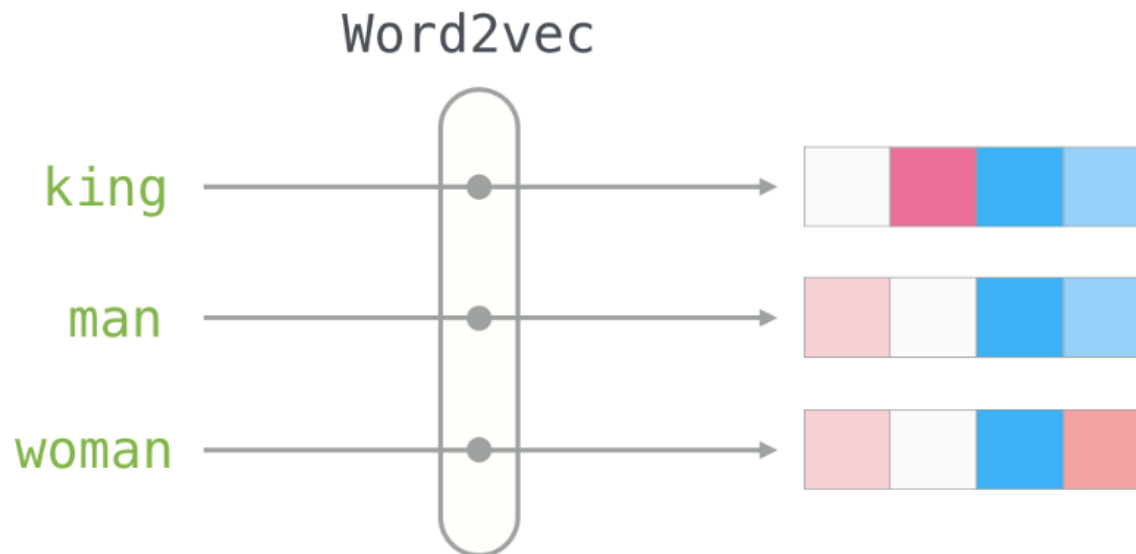
- Word Embeddings
- Transformer Encoder
- Modelo BERT

Roteiro

- Word Embeddings
- Transformer Encoder
- Modelo BERT

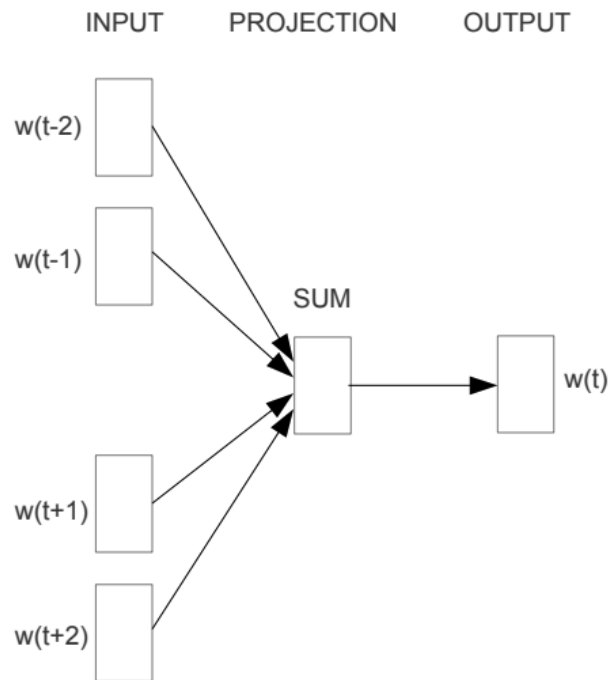
Word Embeddings

- Geramos uma representação densa por palavra

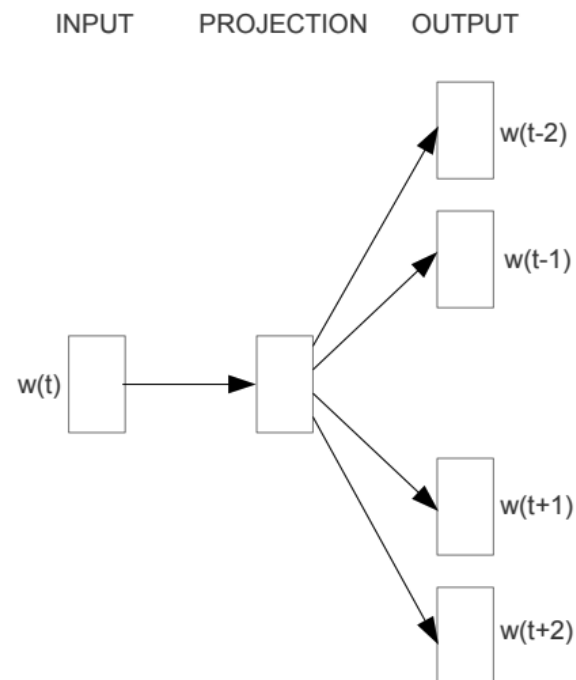


Word Embeddings

- Como fazíamos isso?
 - No Word2Vec clássico, duas alternativas



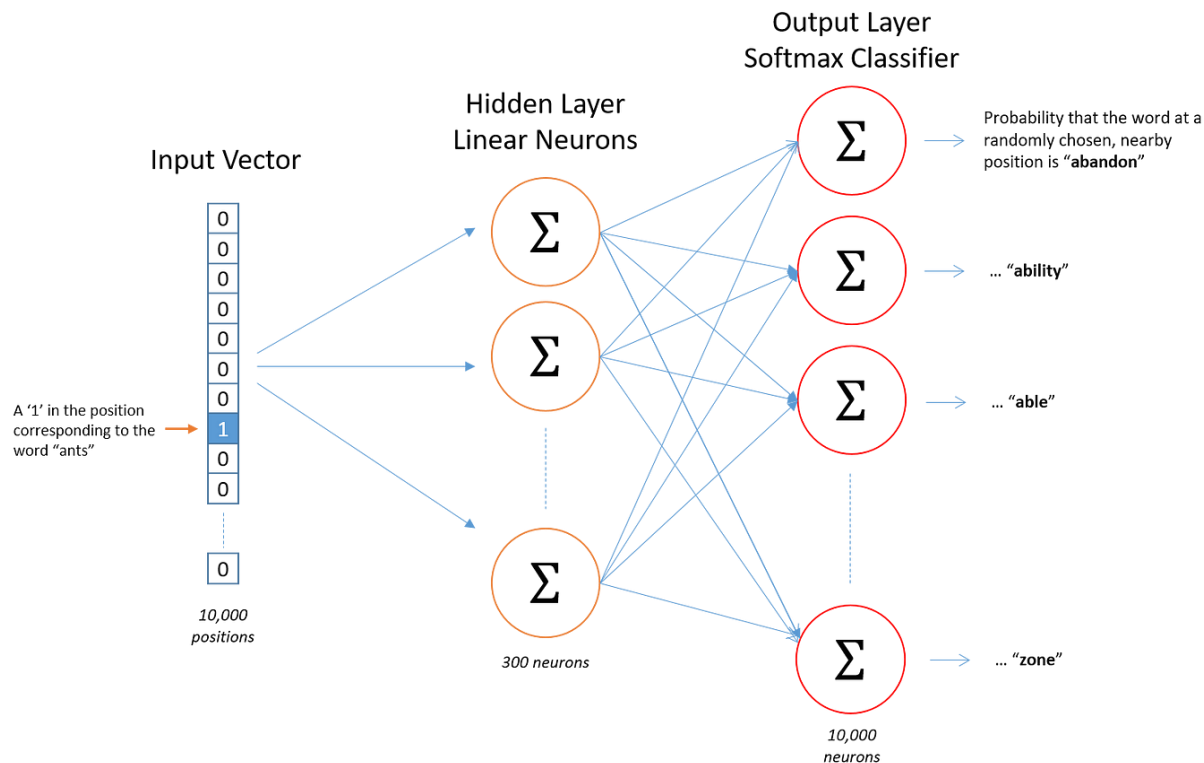
CBOW



Skip-gram

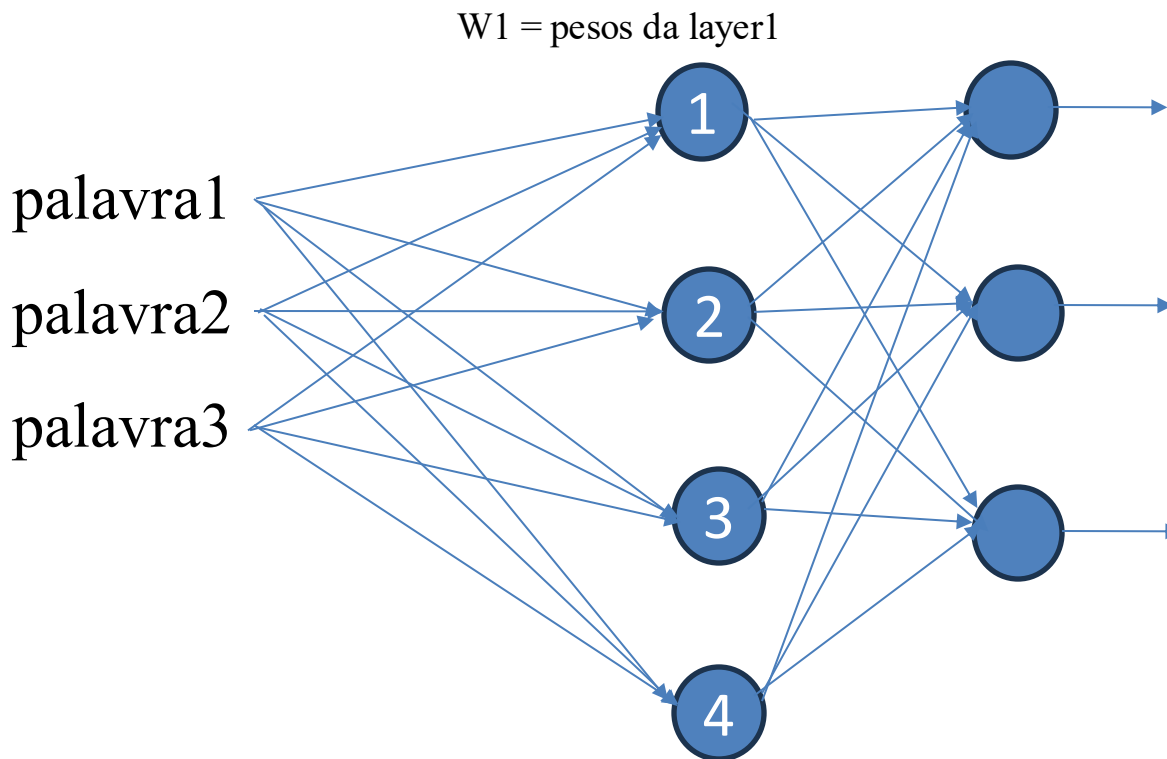
Word Embeddings

- Em ambos os casos temos uma rede neural com uma hidden layer



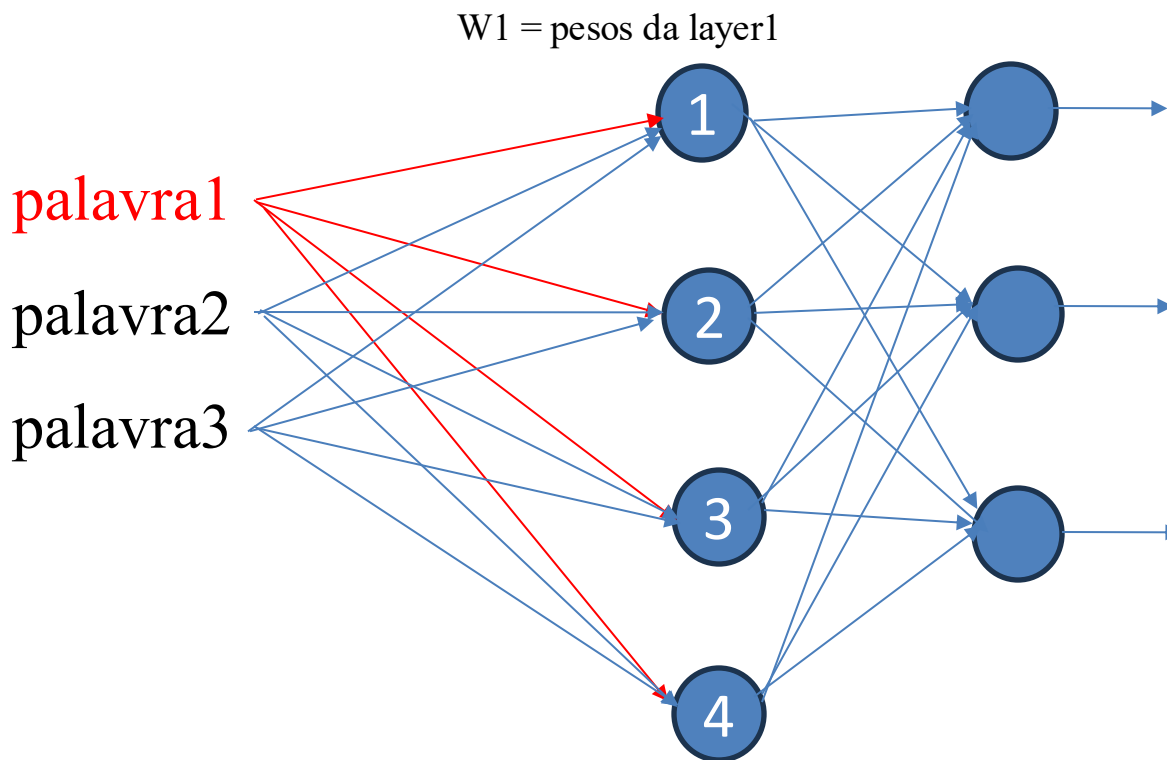
Word Embeddings

- Em ambos os casos temos uma rede neural com uma hidden layer



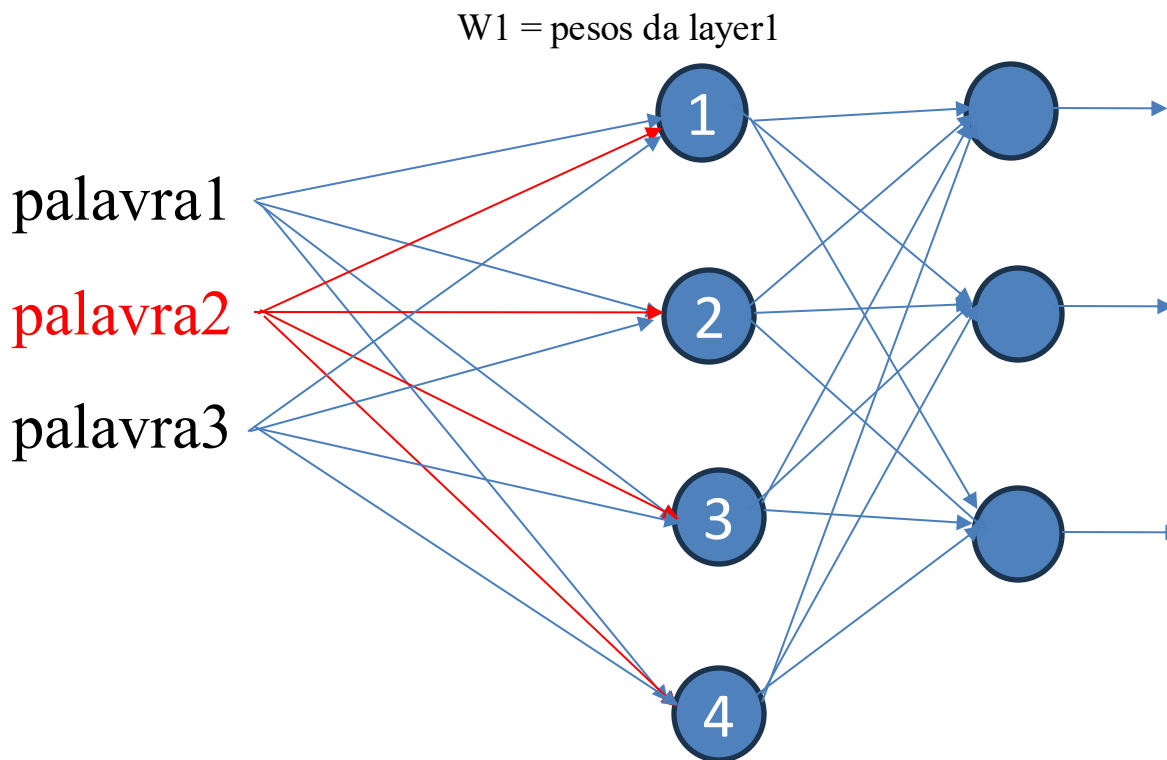
Word Embeddings

- Em ambos os casos temos uma rede neural com uma hidden layer



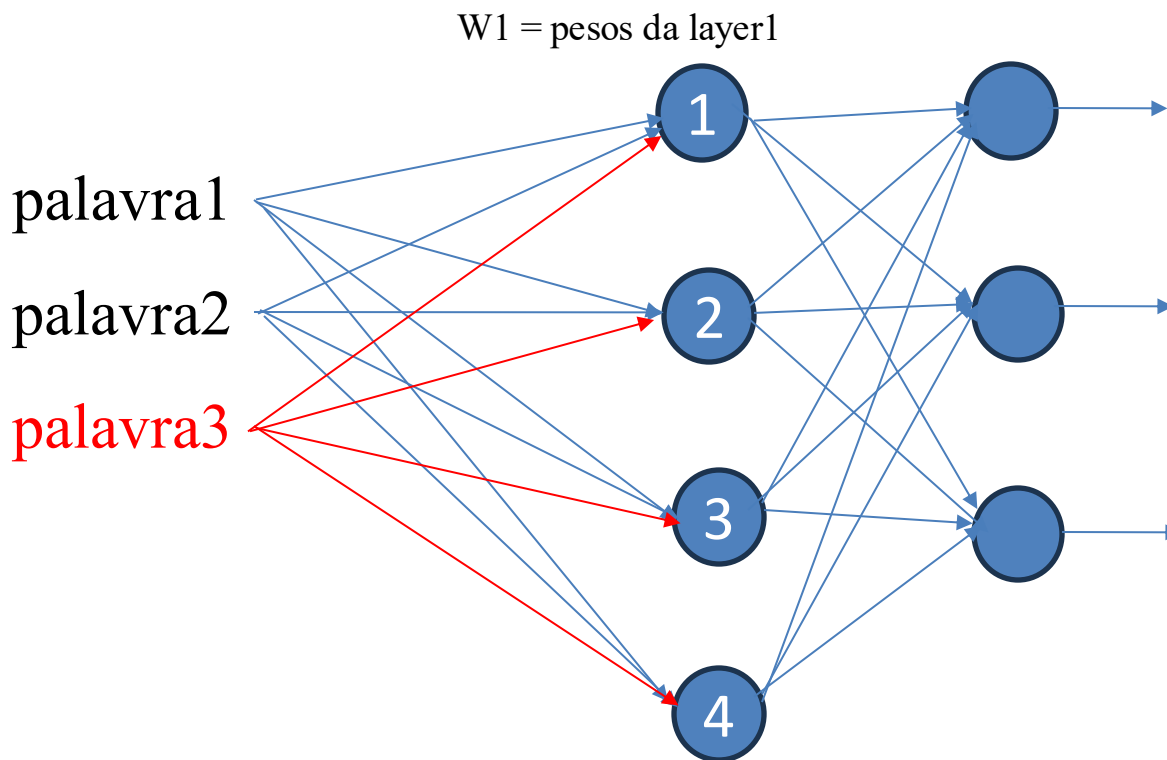
Word Embeddings

- Em ambos os casos temos uma rede neural com uma hidden layer



Word Embeddings

- Em ambos os casos temos uma rede neural com uma hidden layer



Word Embeddings

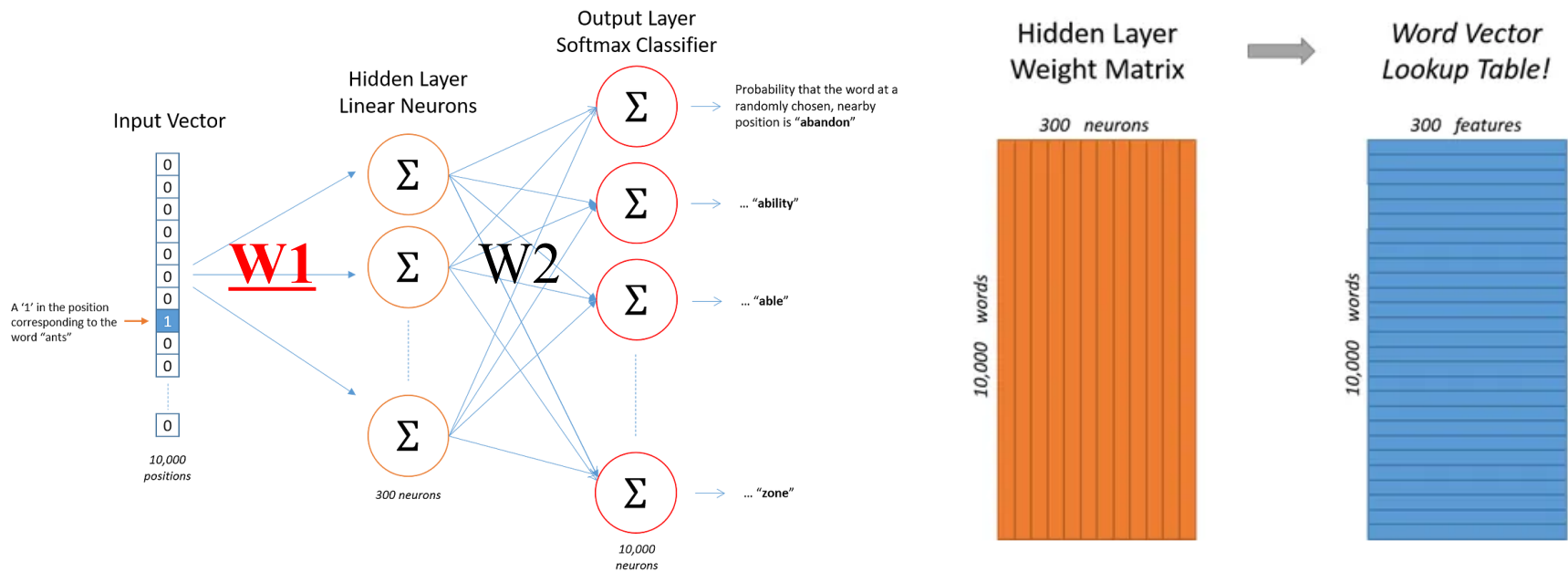
```
class SkipGramModel(nn.Module):
    def __init__(self, vocab_size, embedding_dim):
        super(SkipGramModel, self).__init__()
        self.in_embed = nn.Embedding(vocab_size, embedding_dim)
        self.linear = nn.Linear(embedding_dim, vocab_size)

    def forward(self, target):
        in_embeds = self.in_embed(target)
        out_embeds = self.linear(in_embeds)
        out_embeds = torch.log_softmax(out_embeds, dim=1)
        return out_embeds
```



Camada que mapeia o
vetor one-hot para os
embeddings (pesos
 W_1)

Word Embeddings



Word Embeddings

- O que mapeamos:
 - Significado semântico geral da palavra
 - Componente sintático também



Word Embeddings

- Com Word Embeddings temos:
 - Representações **estáticas** de palavras
 - Principal alternativa de *Transfer Learning* em NLP
 - Usamos *embeddings* já treinados em uma RNN qualquer
 - Melhor que um início aleatório!
- Mas...

Word Embeddings

- Três principais problemas:
 - Como representar documentos?
 - O que fazer com palavras raras ou erros gramaticais?
 - Como lidar com polissemia?

Word Embeddings

- Três principais problemas:
 - Como representar documentos?
 - O que fazer com palavras raras ou erros gramaticais?
 - Como lidar com polissemia?

Word Embeddings

- Como representar documentos?

$$v("O") = \begin{bmatrix} 0.64 & -0.34 & 0.65 & 0.12 & 0.44 & \dots & -0.01 & -0.32 & 0.09 & 0.15 \end{bmatrix}$$

+

$$v("cachorro") = \begin{bmatrix} 0.34 & -0.12 & 0.01 & -0.89 & 0.33 & \dots & 0.82 & 0.37 & -0.91 & 0.65 \end{bmatrix}$$

+

$$v("correu") = \begin{bmatrix} 0.26 & 0.04 & -0.51 & -0.24 & -0.51 & \dots & -0.77 & -0.20 & 0.29 & 0.98 \end{bmatrix}$$

/3 =

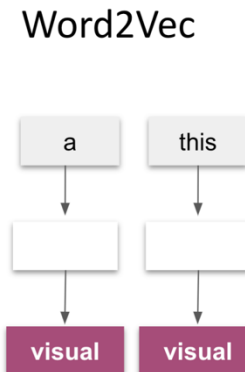
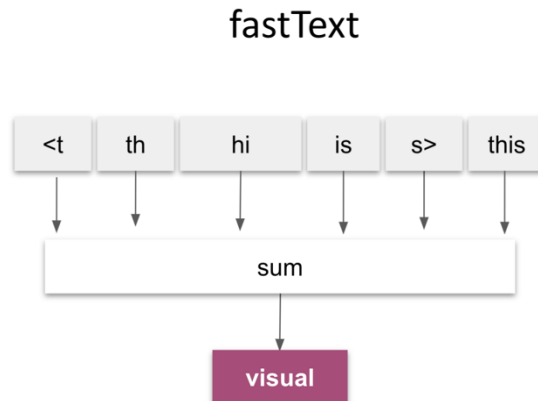
$$v("O cachorro correu") = \begin{bmatrix} 0.41 & -0.14 & 0.05 & -0.33 & 0.08 & \dots & 0.01 & -0.05 & -0.17 & 0.59 \end{bmatrix}$$

Word Embeddings

- Três principais problemas:
 - ~~Como representar documentos?~~
 - O que fazer com palavras raras ou erros gramaticais?
 - Como lidar com polissemia?

Word Embeddings

- O que fazer com palavras raras ou erros gramaticais?
 - Na prática, podemos usar outros algoritmos de embeddings
 - Principal exemplo: FastText



Word Embeddings

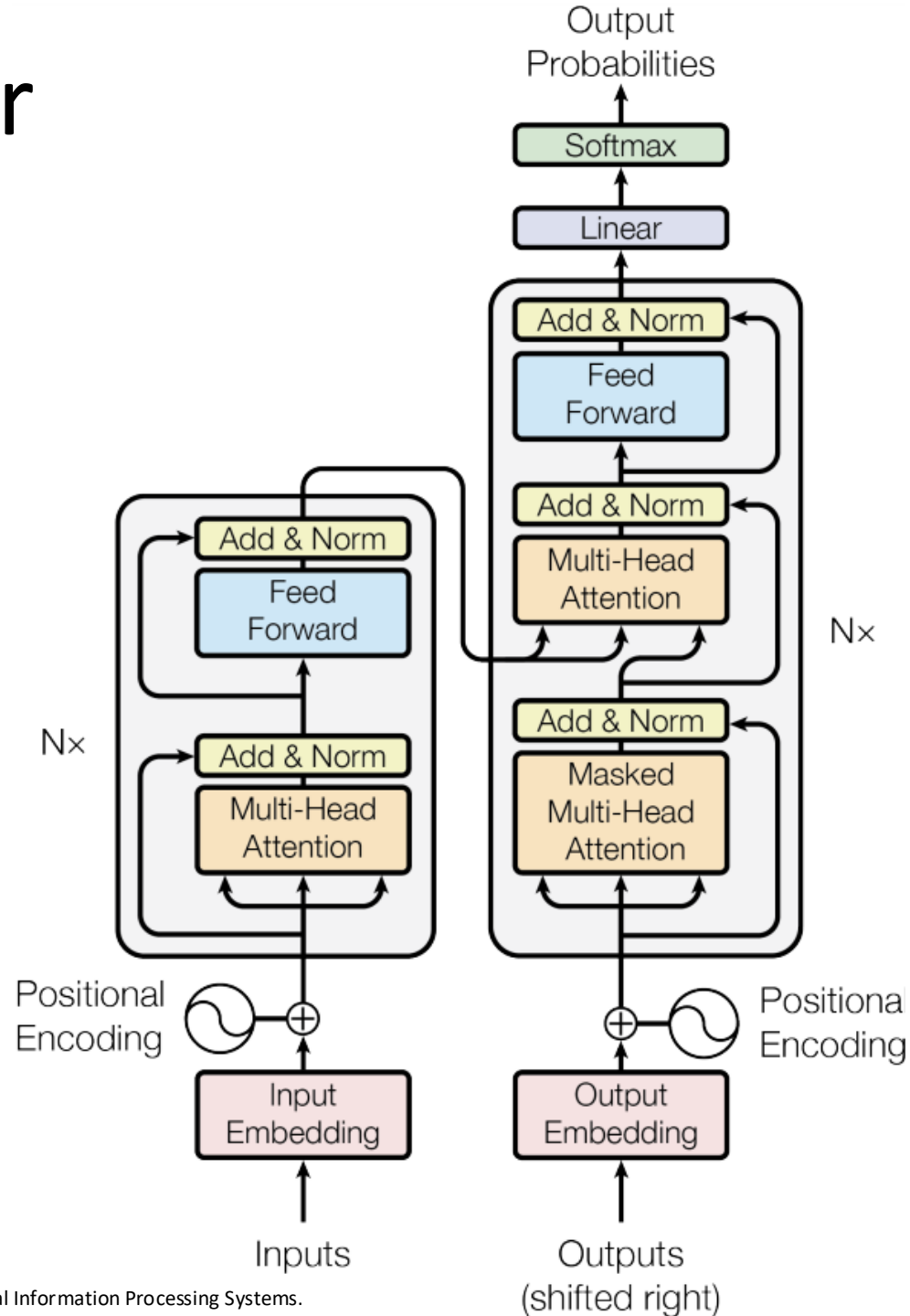
- Três principais problemas:
 - ~~Como representar documentos?~~
 - ~~O que fazer com palavras raras ou erros gramaticais?~~
 - Como lidar com polissemia?
 - Não lidamos
 - Ainda...



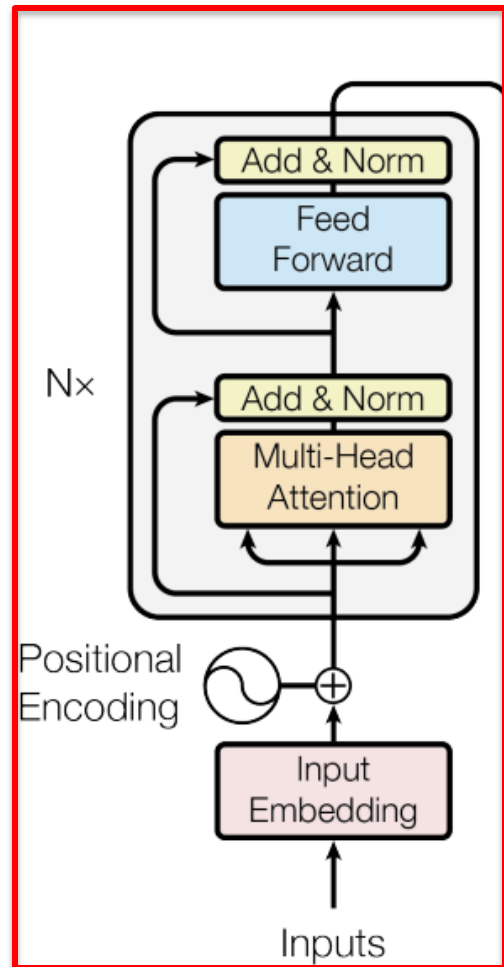
Roteiro

- Word Embeddings
- Transformer Encoder
- Modelo BERT

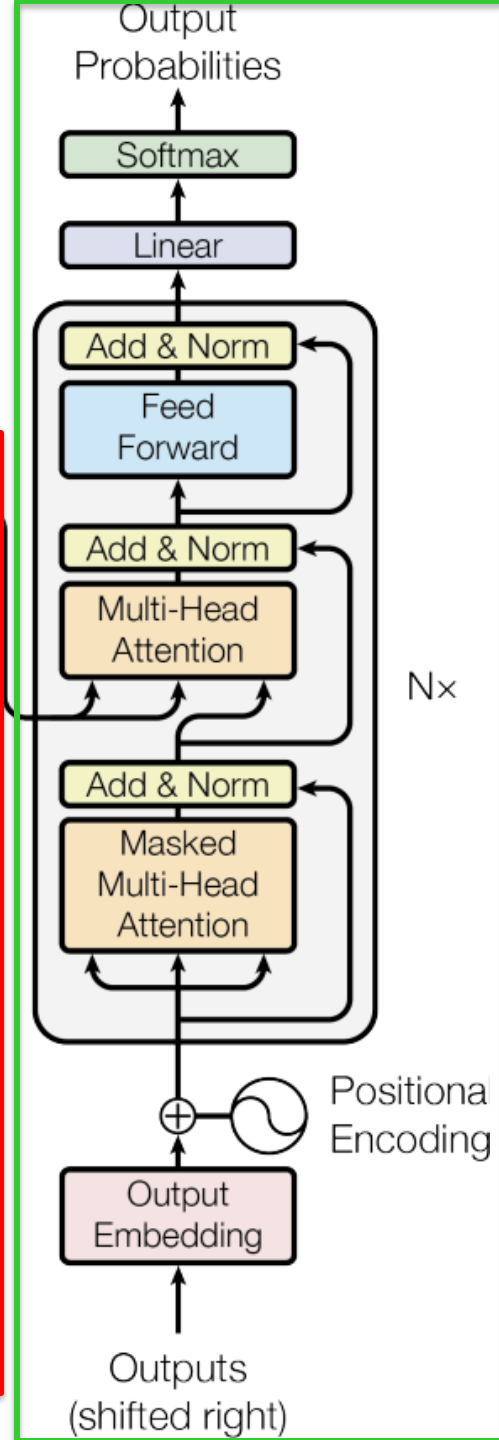
Transformer Encoder



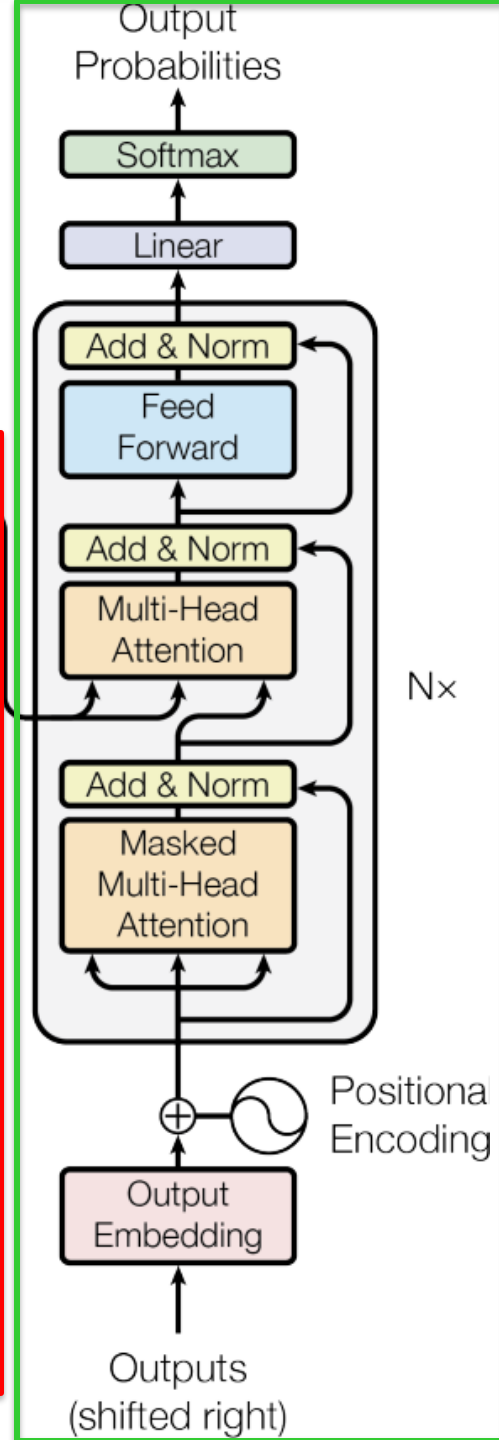
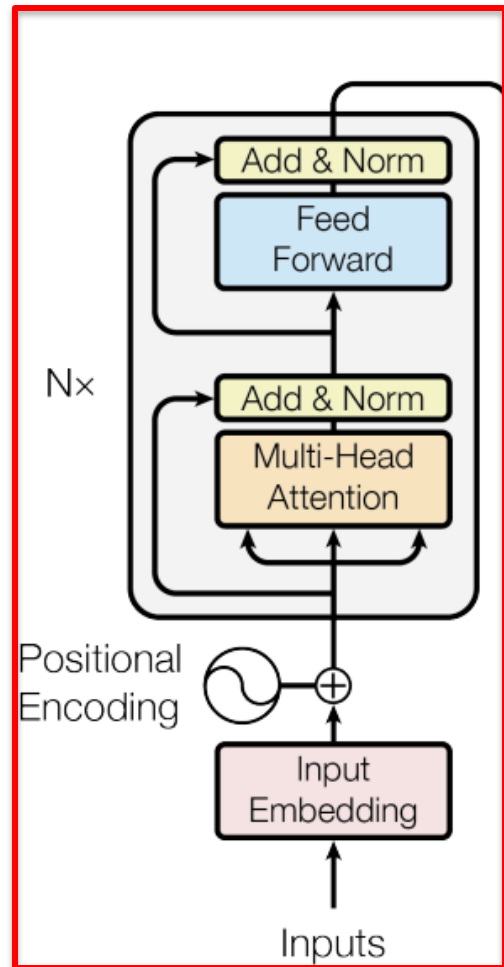
Encoder



Decoder



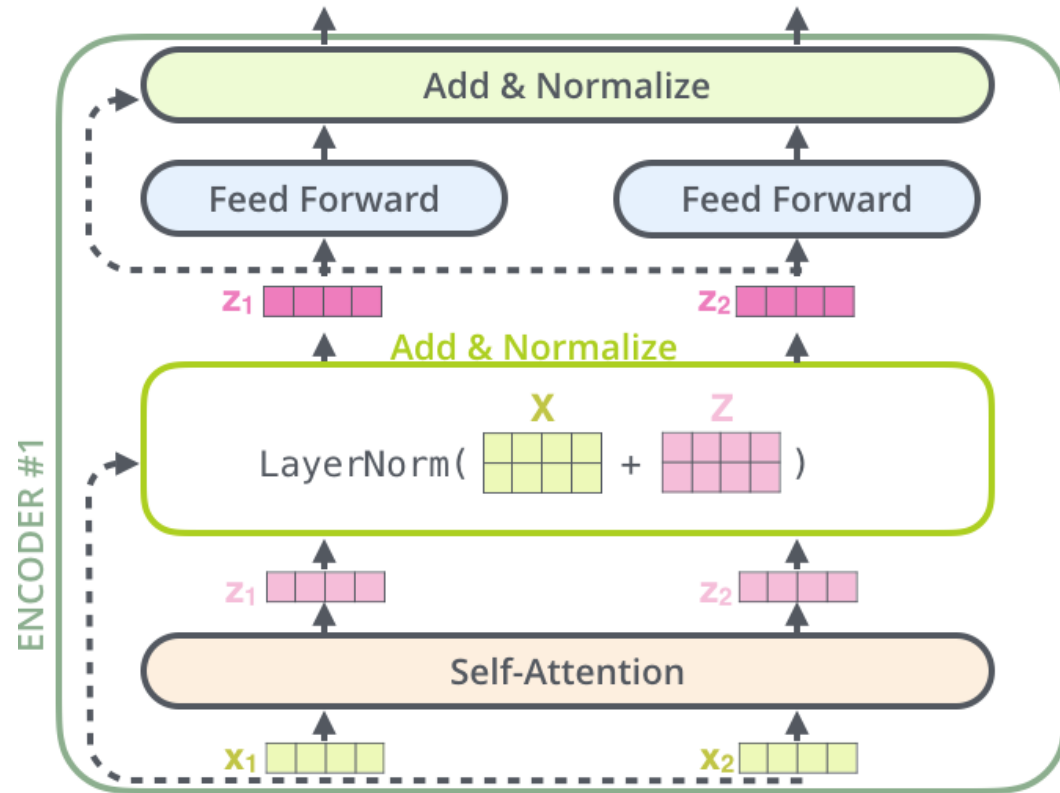
Encoder
Compreender o
Texto



Decoder
Gerar Texto
Novo

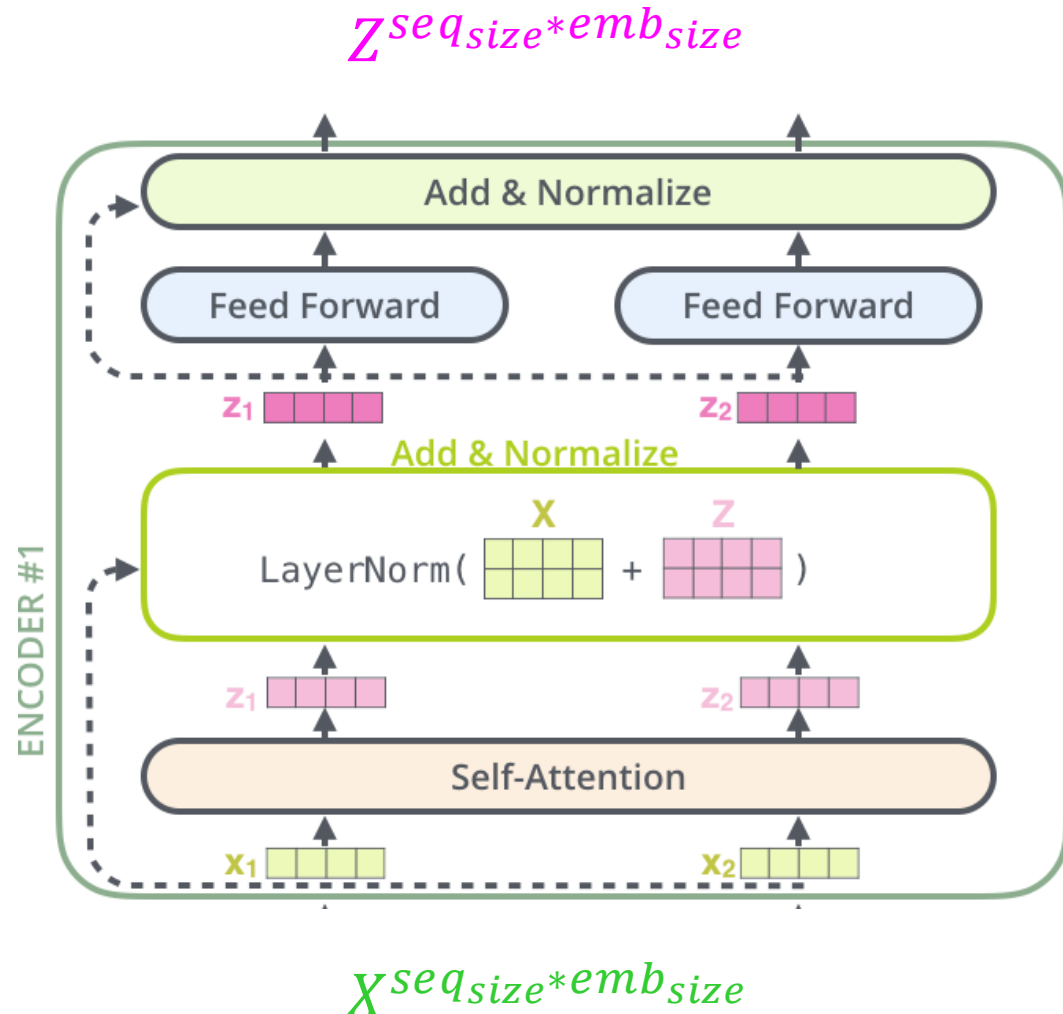
Transformer Encoder

- Em uma rede temos vários blocos de encoder
- O que acontece em um bloco?



Encoder

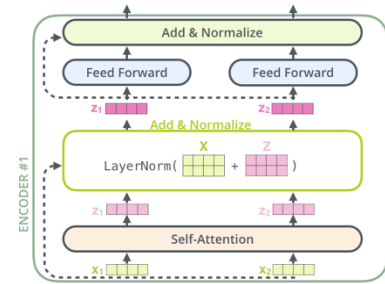
- Em uma rede temos vários blocos de encoder
- O que acontece em um bloco?
 - Temos uma **representação** para **cada token** de entrada



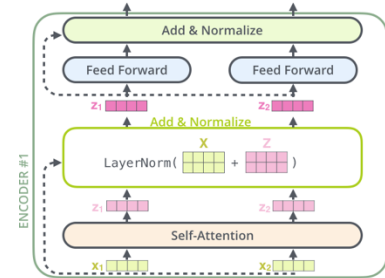
Encoder

- Em uma rede temos vários blocos de encoder
- O que acontece em um bloco?
 - Temos uma **representação** para **cada token** de entrada
 - Ao final de cada uma das camadas de encoder

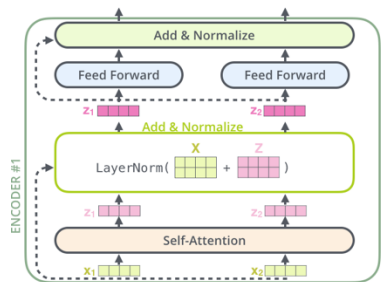
$Z_3^{seq_size * emb_size}$



$Z_2^{seq_size * emb_size}$



$Z_1^{seq_size * emb_size}$



$X^{seq_size * emb_size}$

Encoder

- Perceba que após cada camada de encoder, temos **uma representação para cada palavra**
- Por conta do *self-attention* essa representação considera as palavras ao redor
- Ou seja: temos uma **representação contextual** para cada palavra!
- E assim surge...

BERT

- “Bi-directional Encoder Representations from Transformers”



Roteiro

- Word Embeddings
- Transformer Encoder
- Modelo BERT

BERT

- Lançado em 2018 pela equipe da Google
- Baseado em **Pré-treino** e *Fine-tuning*
- Objetivo é criar **representações contextuais** para sentenças e palavras



BERT

- Ideia geral:
 - “Se tivermos *boas representações* para as palavras, podemos resolver *qualquer problema* que envolva a *compreensão* da linguagem.”



BERT

- O objetivo:
 - Ter uma **representação contextual** de cada palavra
- O problema:
 - Como?
 - Transformer original era treinado para tradução

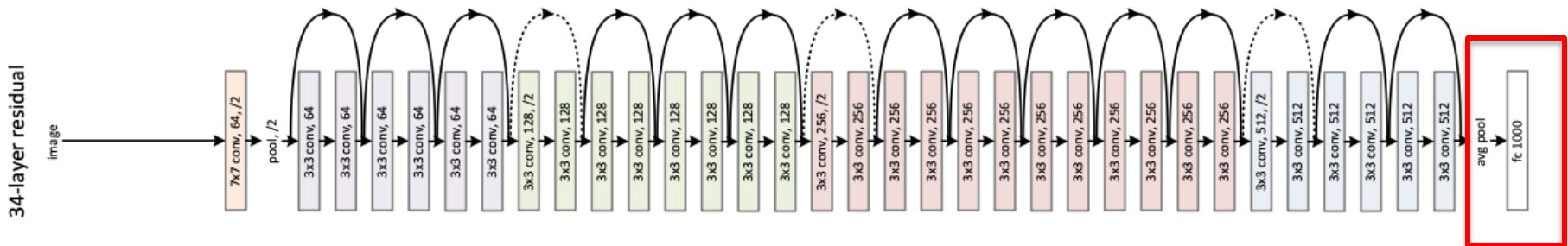


BERT

- Anteriormente estávamos lidando com tradução
- Entrada -> Sentença original
- Saída -> Sentença traduzida
- Se quisermos usar só o *encoder*, nossa saída é uma matriz da mesma dimensão de entrada

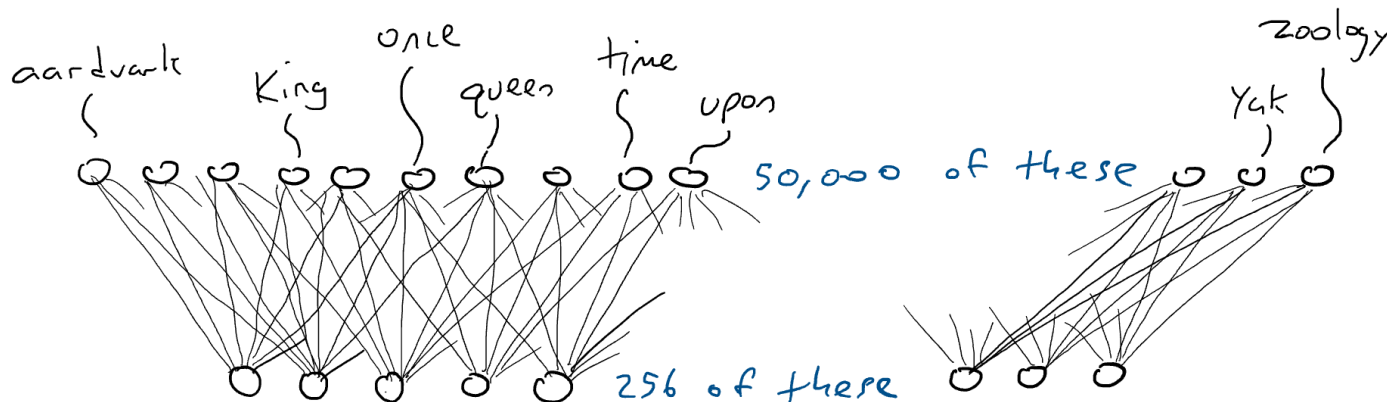
BERT

- Quando queríamos criar representações de uma imagem, usávamos uma tarefa de **classificação**



BERT

- Para NLP temos uma alternativa interessante: **modelagem de linguagem**
- Na prática é uma classificação onde **nosso rótulo é a próxima palavra**
- Tarefa *self-supervised*



BERT

- Problema:
 - LM clássico é feio de forma contínua/auto-regressiva
 - Para cada palavra t :

$$P(y_1, y_2, \dots, y_t) \\ = P(y_1) * P(y_2|y_1) * \dots * P(y_t|y_1) * P(y_t|y_1, y_2, \dots, y_{t-1})$$

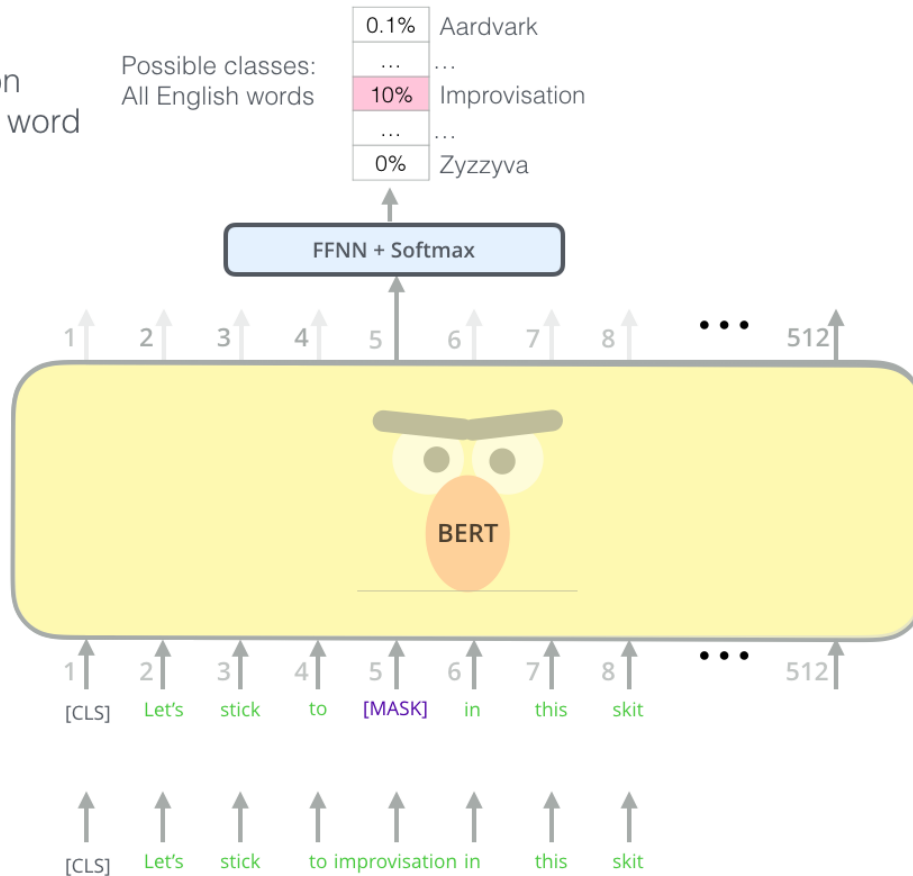
- Na prática a palavra no tempo 2 só sabe da existência da palavra 1

BERT

- Resolvemos com a primeira tarefa de pré-treino: *Masked Language Modeling (MLM)*
 - Mascaramos algumas palavras que entram no modelo
 - Forçamos o modelo a descobrir qual palavra foi mascarada

BERT

Use the output of the masked word's position to predict the masked word



BERT

- Sobre MLM:
 - 15% dos tokens da **sentença de entrada** são selecionados para predição:
 - 80% das vezes eles são mascarados (**[MASK]**)
 - 10% das vezes eles são trocados por **outras palavras**
 - 10% das vezes eles são **mantidos**
 - Por que essas diferentes abordagens?

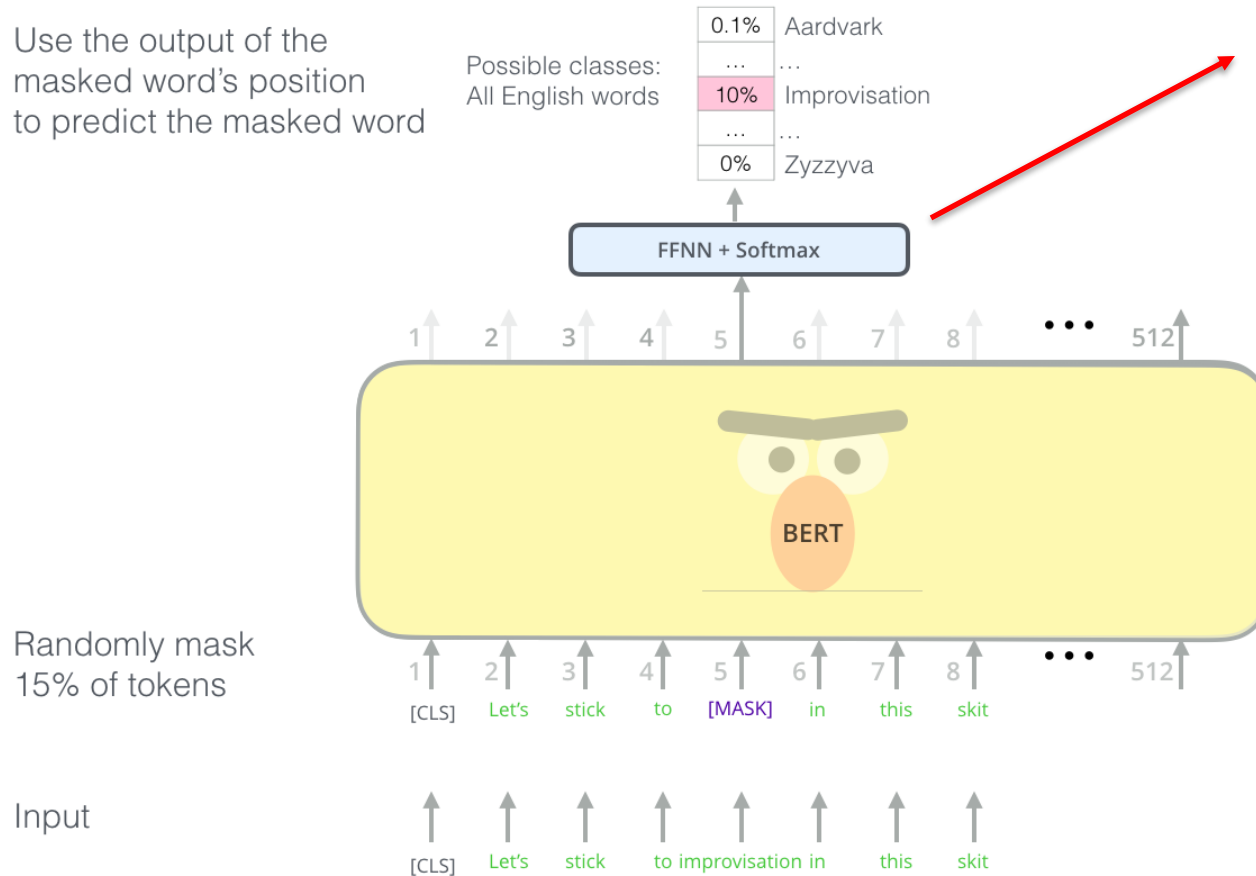
BERT

- Sobre MLM:
 - 15% dos tokens da **sentença de entrada** são selecionados para predição:
 - 80% das vezes eles são mascarados (**[MASK]**)
 - 10% das vezes eles são trocados por **outras palavras**
 - 10% das vezes eles são **mantidos**
 - Por que essas diferentes abordagens?
 - Quando formos fazer o *fine-tuning*, não será visto o token **[MASK]**!
 - Ao final, usamos *Cross-Entropy* para treinar o modelo

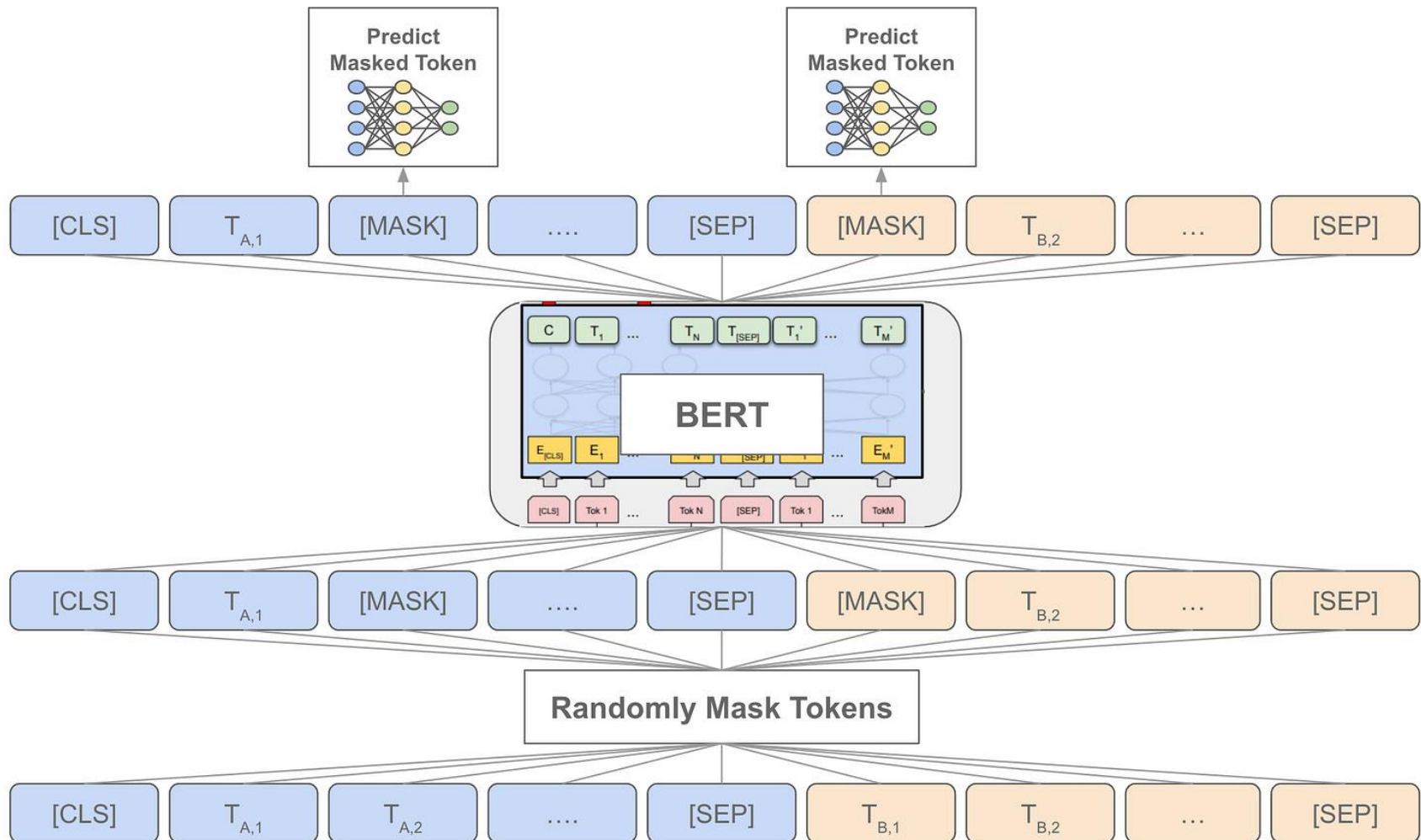
BERT

Cada token
seleccionado é passado
para uma camada
linear com softmax e
predito

Use the output of the
masked word's position
to predict the masked word



BERT



BERT

- Grande vantagem da combinação de **MLM** e *self-attention* é permitir que um token seja previsto com base em **toda a sentença**, tanto tokens anteriores quanto posteriores!

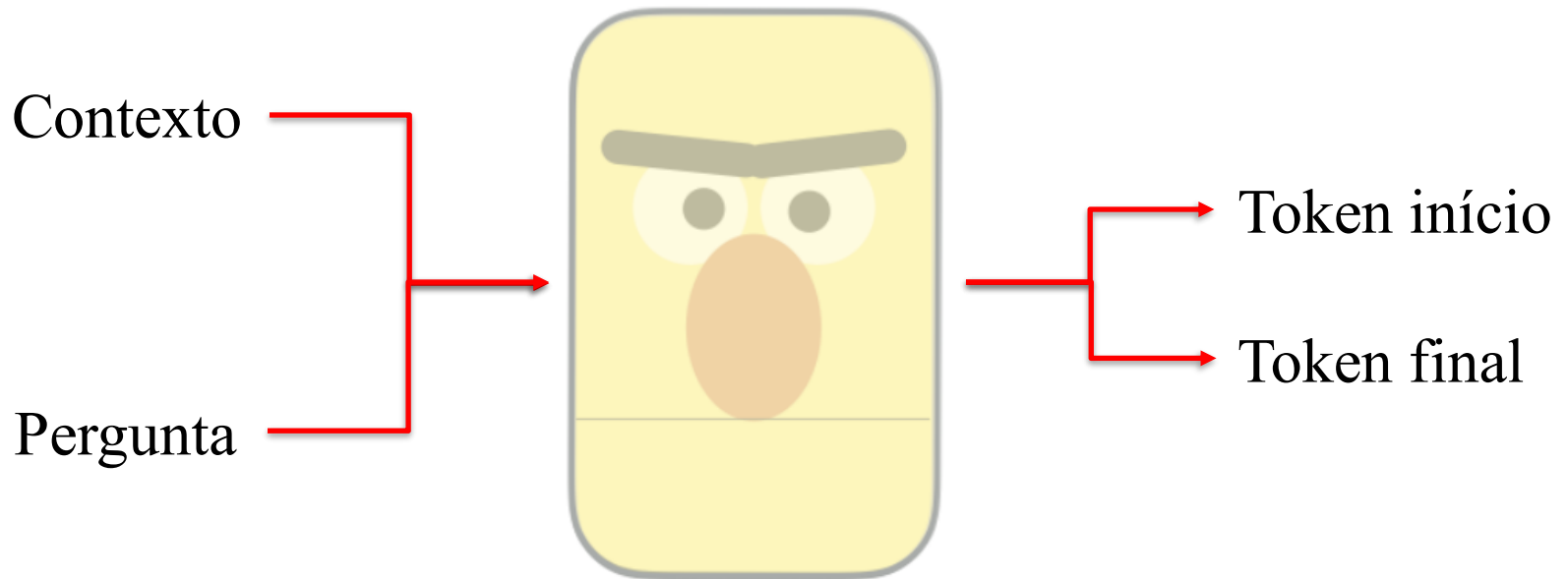


BERT

- Todavia, MLM não é a única tarefa de pré-treino do modelo
- NLP inclui tarefas com diferentes entradas e saídas:
 - NER
 - Classificação
 - Tradução
 - Question & Answering

BERT

- Tarefas como Q&A recebem duas sentenças



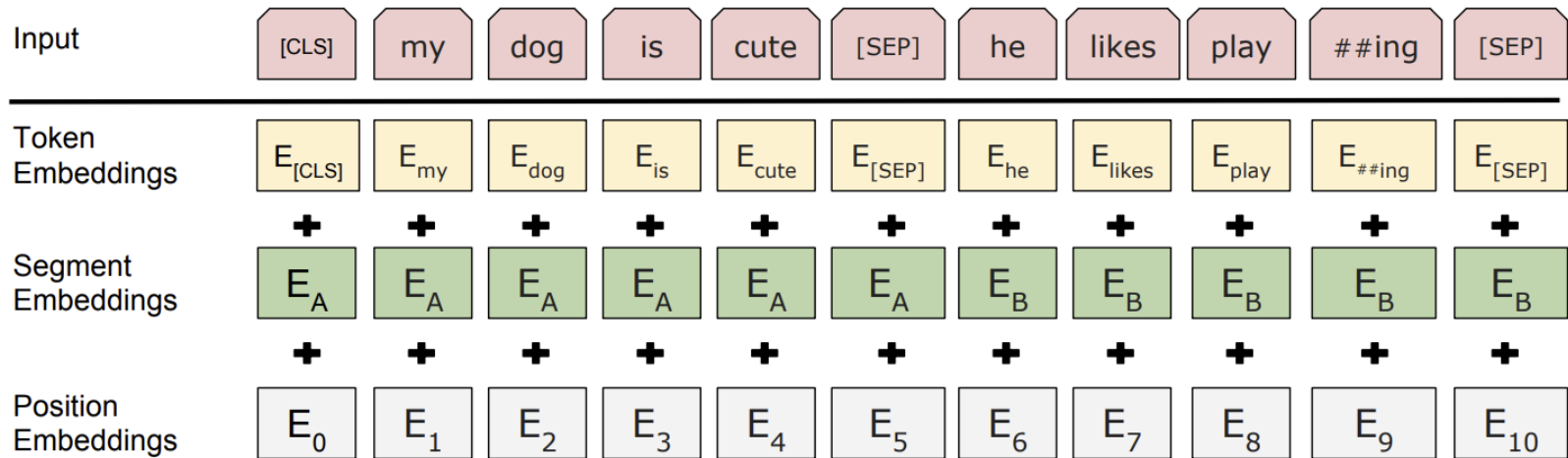
- Como relacionar elas?

BERT

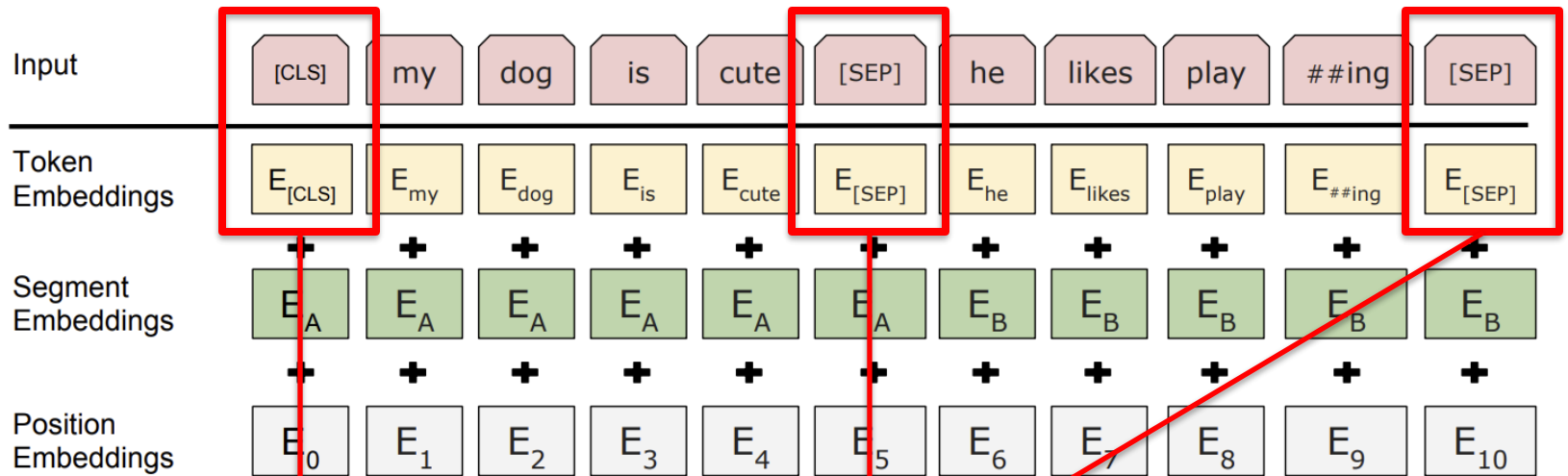
- Next-sentence prediction (NSP)
 - Fornecemos duas sentenças ao BERT
 - Ensinaamos ele a predizer quando a **sentença A é diretamente seguida da sentença B**
 - Tarefa binária: *IsNext* e *NotNext*
 - 50% das vezes a sentença era trocada
- Como saber quando estamos passando uma sentença ou outra?

BERT

- Mais um embedding:



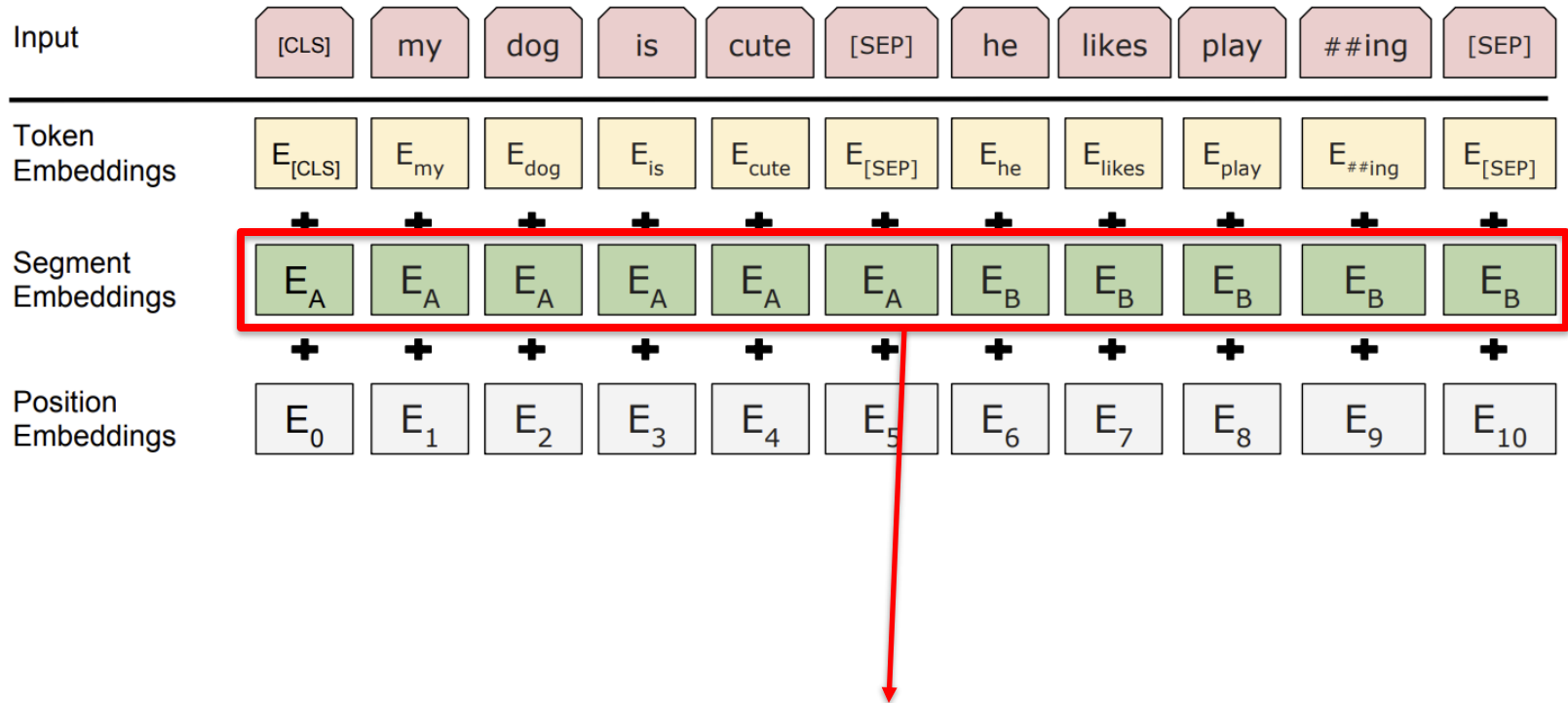
BERT



Temos um token especial `[CLS]`, usamos ele para classificar NSP

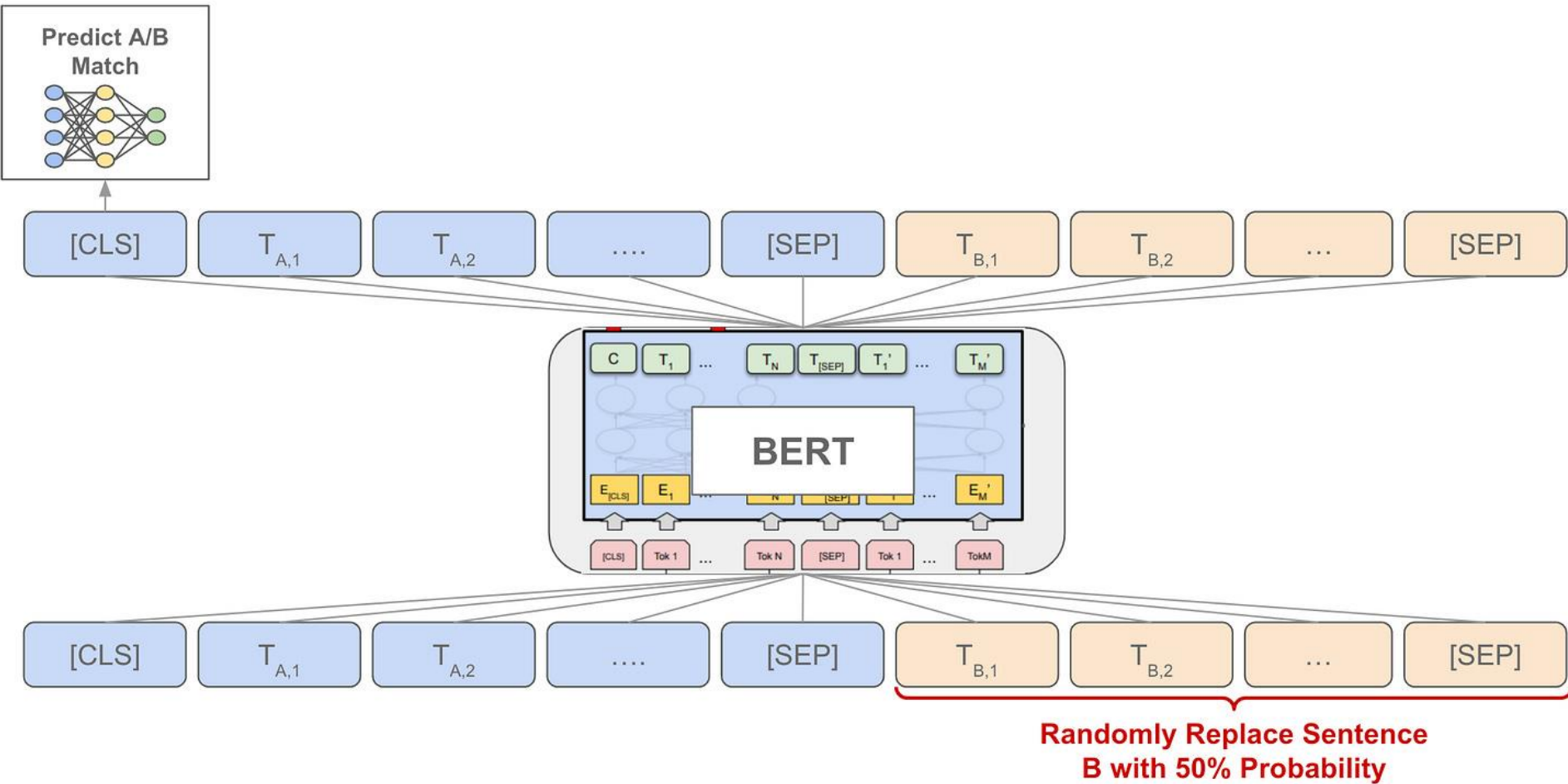
Temos um token especial `[SEP]`, usamos para indicar quando termina uma sentença

BERT



Adicionamos um novo “Positional embedding”,
indicando quando temos uma ou outra sentença.
Na prática, 0 e 1.

BERT



BERT

- Datasets:
 - BookCorpus
 - 800 milhões de palavras
 - Livros gratuitos e de domínio público disponíveis na internet
 - English Wikipedia
 - 2.5 bilhões de palavras
 - Todas as páginas da wikipedia escritas em inglês
 - Apenas texto!

BERT

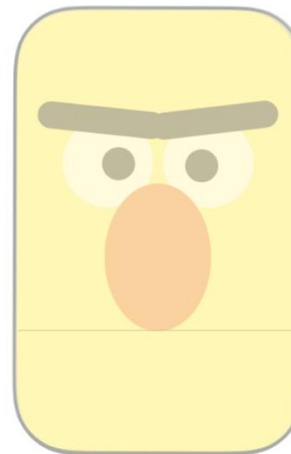
- Pré-treino:
 - Entrada de 512 tokens durante treinamento
 - Batch size de 256 sequências
 - 1.000.000 steps (~40 épocas)
 - Adam
 - LR de $1e-4$
 - Warmup de 10.000 steps
 - Dropout 0.1 em todas as camadas
 - Função de ativação GeLU

BERT

- Pré-treino:
 - $BERT_{BASE}$ - 110M de parâmetros - 16 TPUs por 4 dias
 - Mesmo tamanho do GPT (veremos semana que vem)
 - $BERT_{LARGE}$ - 340M de parâmetros - 64 TPUs por 4 dias



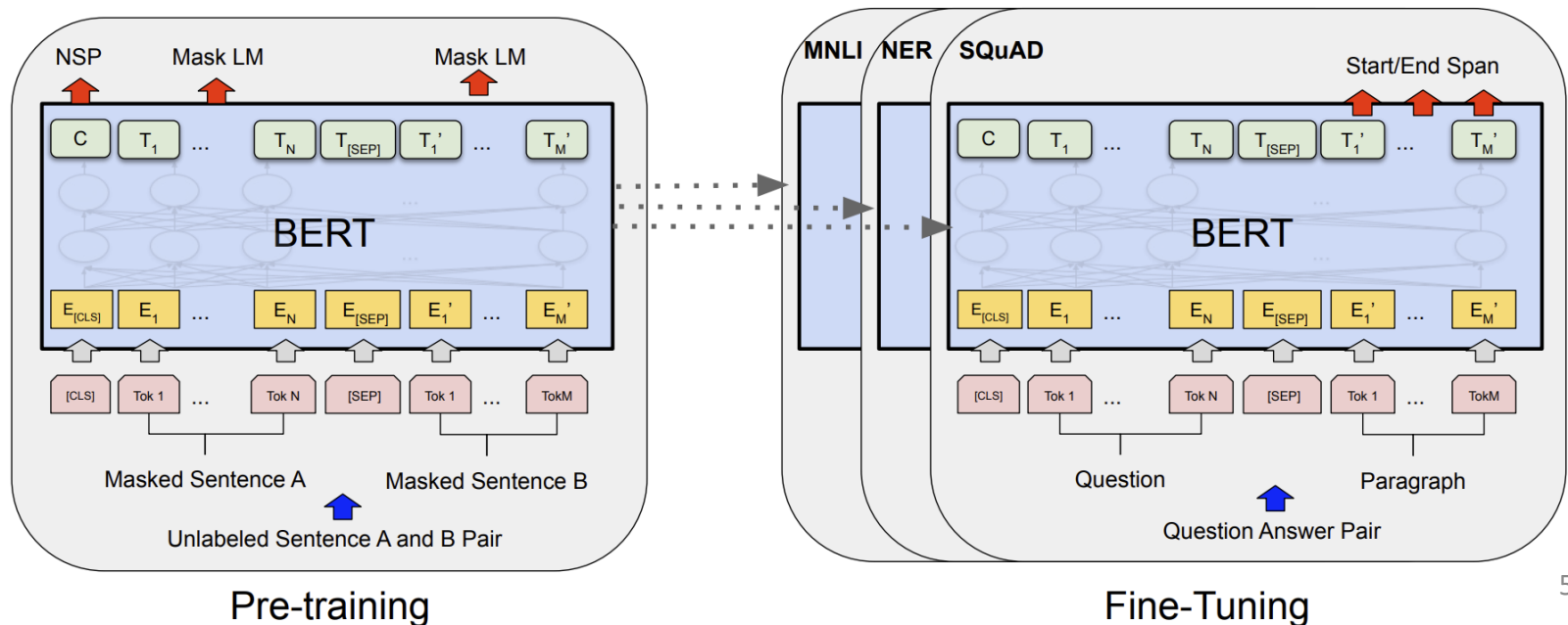
BERT_{BASE}



BERT_{LARGE}

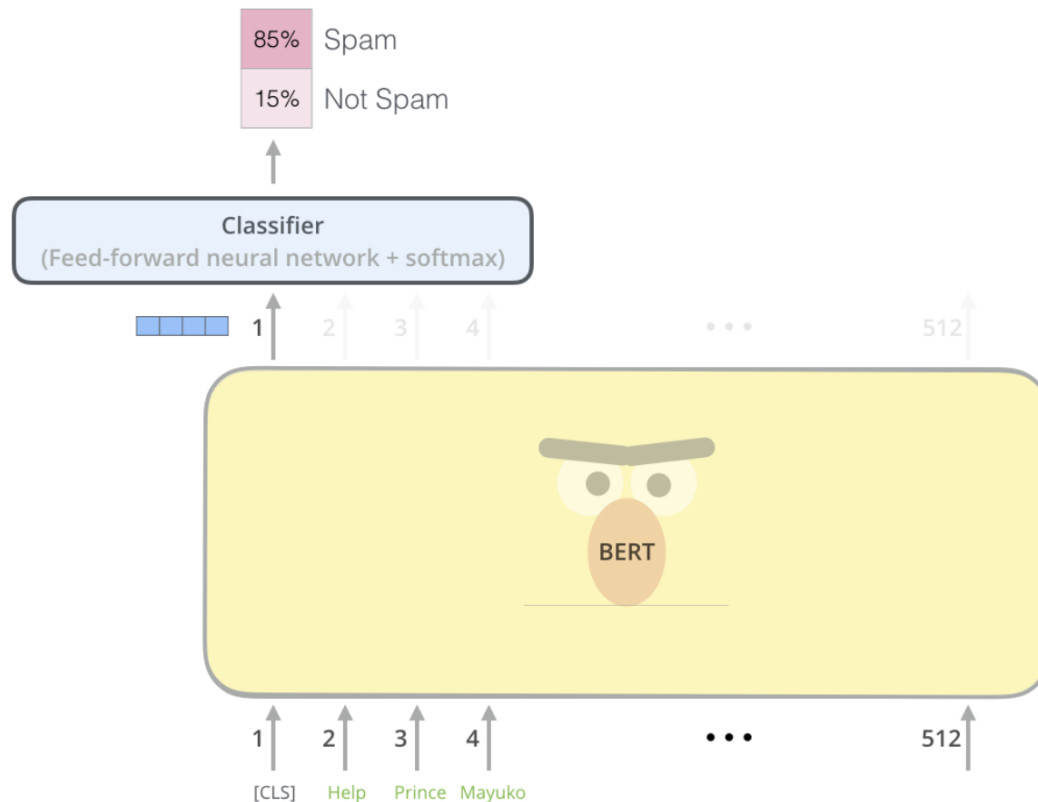
BERT

- Com o modelo já treinado, podemos ajustar o BERT para outras tarefas:
 - *Fine-tuning* bem **menos custoso que o pré-treino** original (1 hora em TPU no paper)!



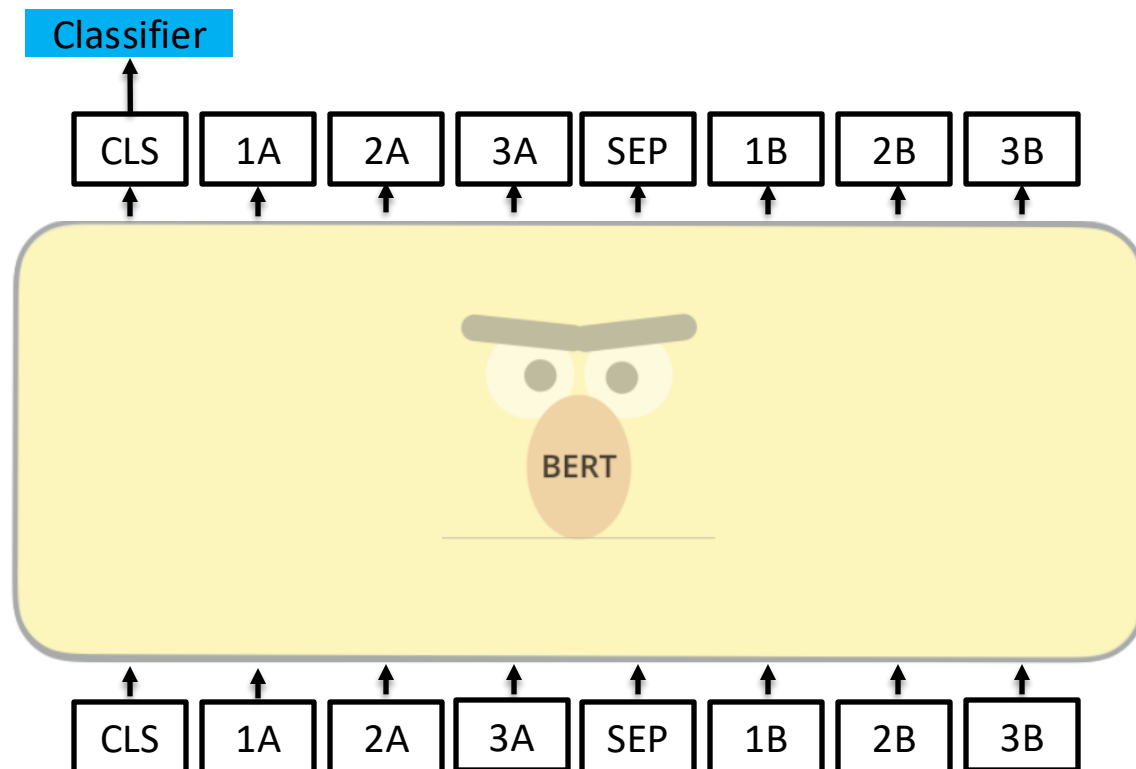
Classificação

- Usamos o token [CLS] para representar a sentença completa



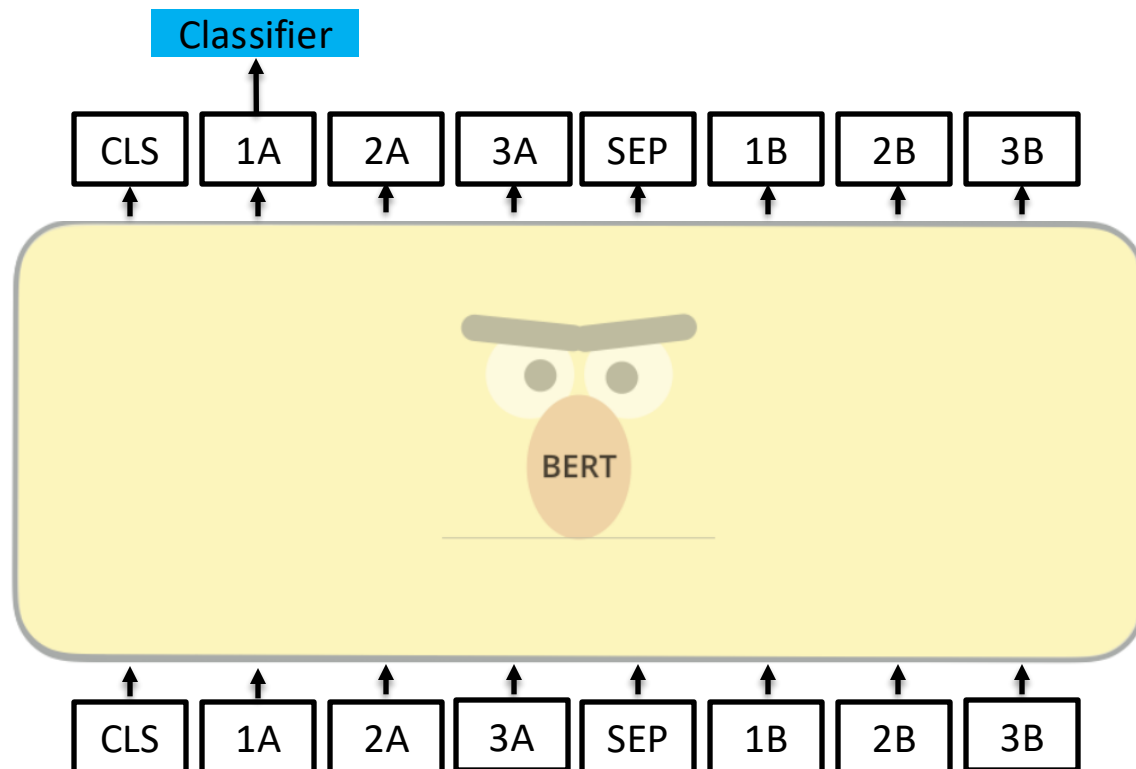
Classificação Nível de Token

- Usamos uma camada linear, aplicada para cada *embedding* de saída



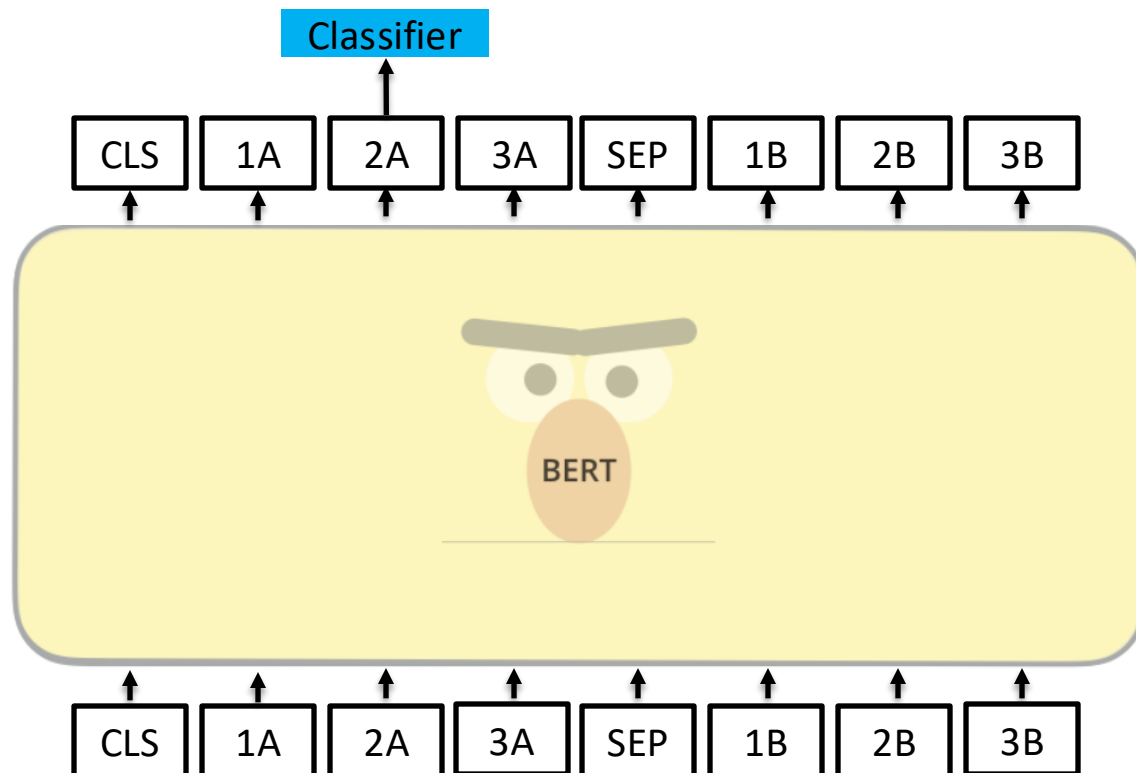
Classificação Nível de Token

- Usamos uma camada linear, aplicada para cada *embedding* de saída

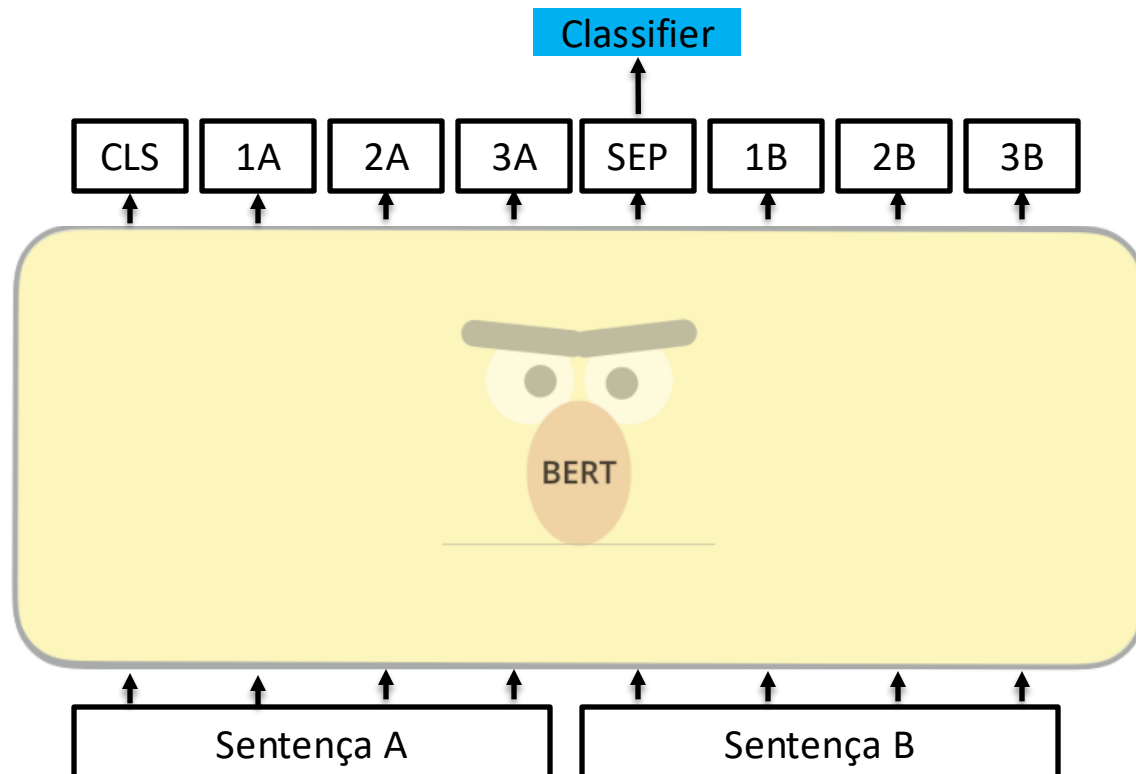


Classificação Nível de Token

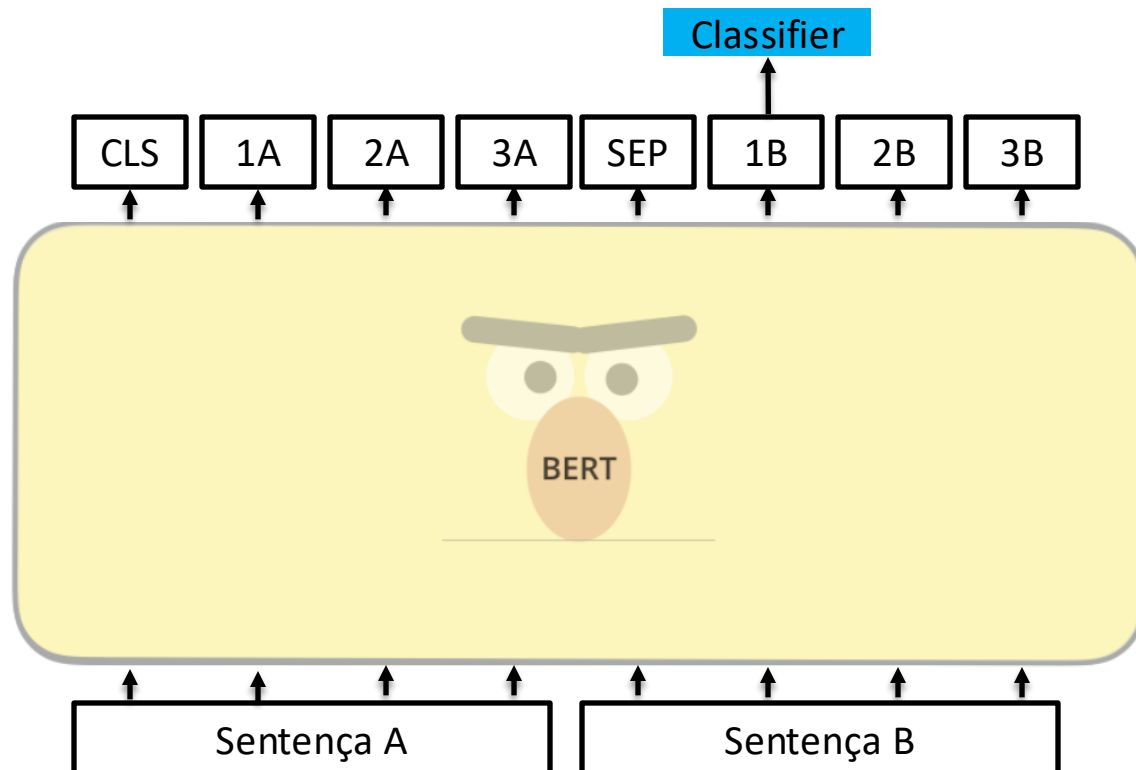
- Usamos uma camada linear, aplicada para cada *embedding* de saída



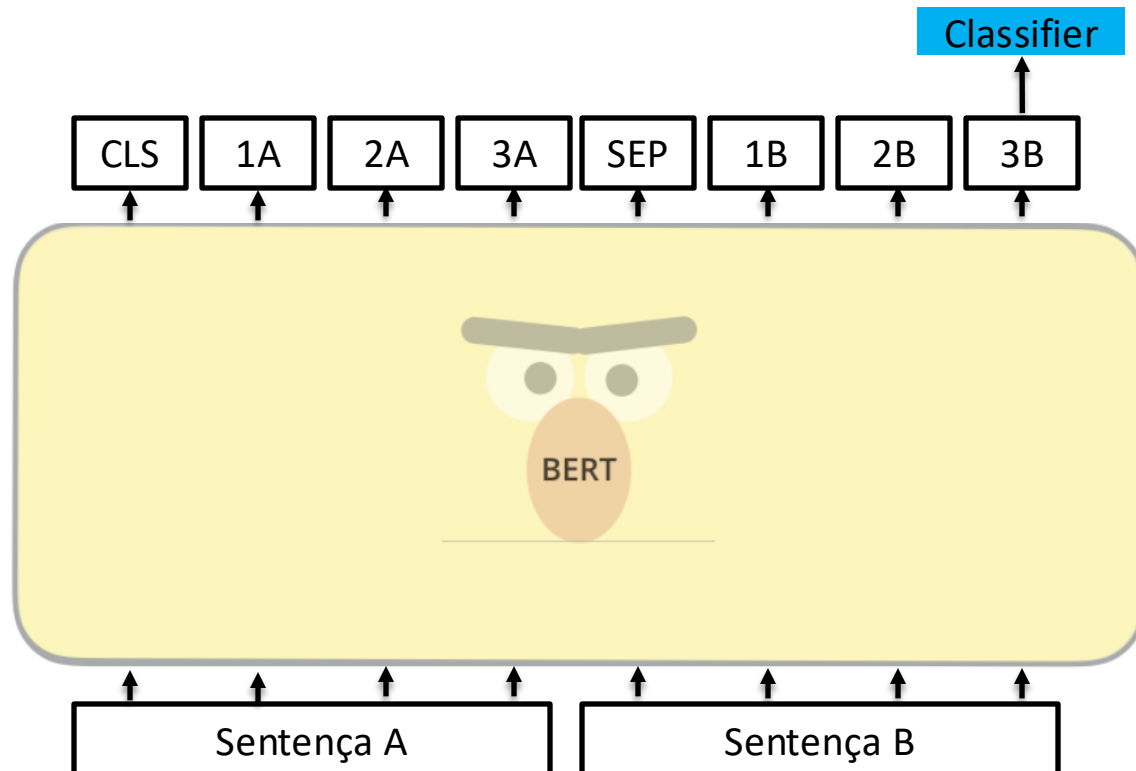
Q&A

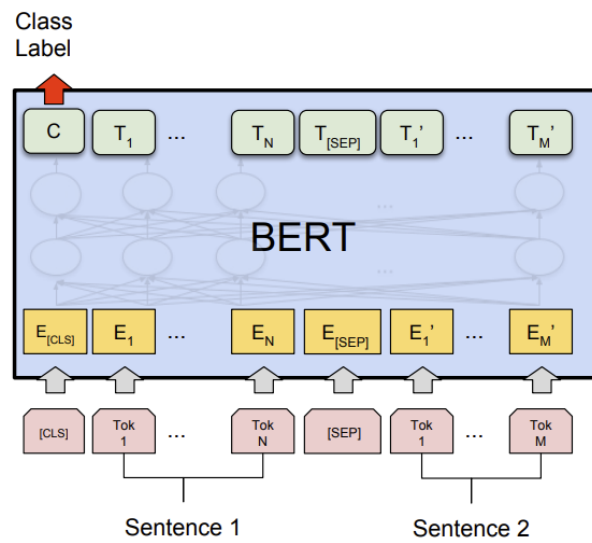


Q&A

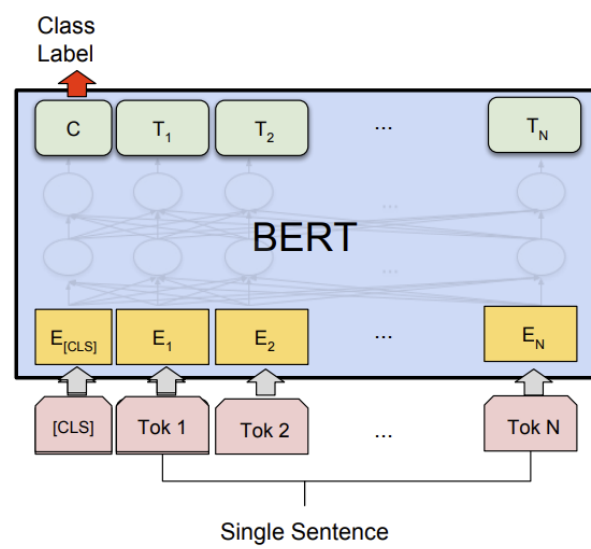


Q&A

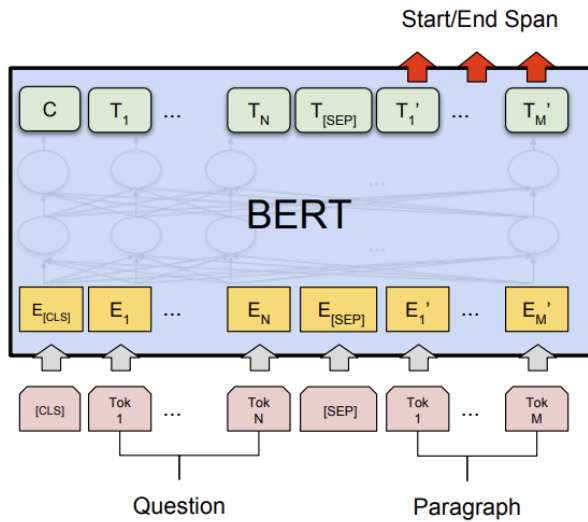




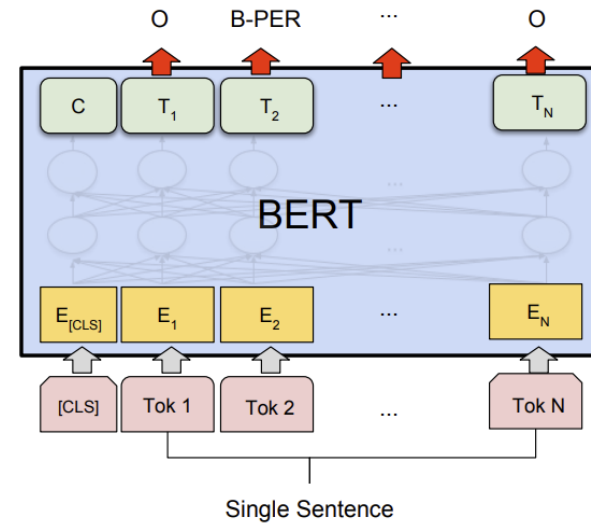
(a) Sentence Pair Classification Tasks:
MNLI, QQP, QNLI, STS-B, MRPC,
RTE, SWAG



(b) Single Sentence Classification Tasks:
SST-2, CoLA



(c) Question Answering Tasks:
SQuAD v1.1

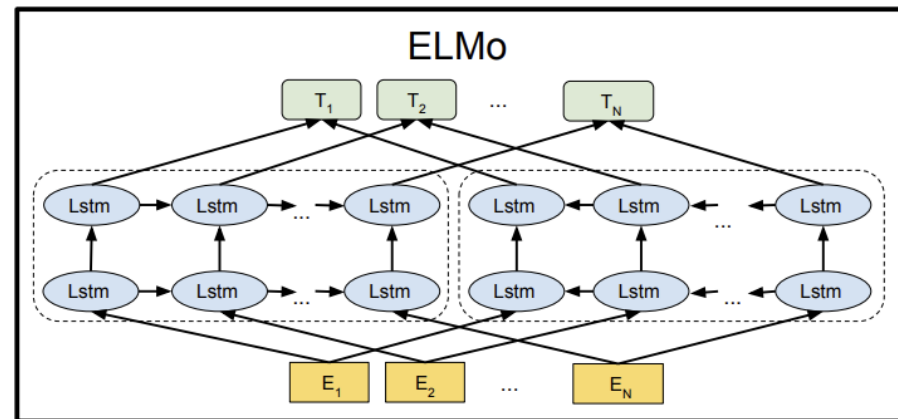
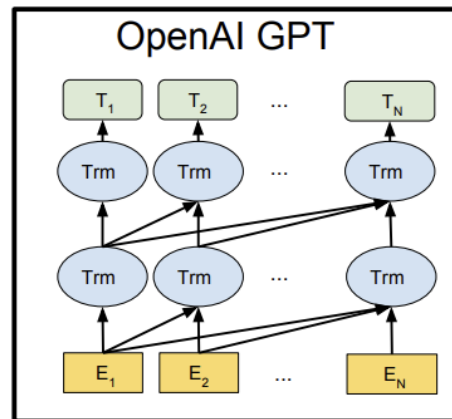
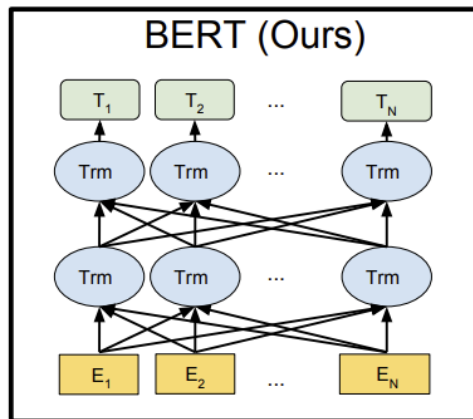


(d) Single Sentence Tagging Tasks:
CoNLL-2003 NER

Figure 4: Illustrations of Fine-tuning BERT on Different Tasks.

BERT

- No artigo original comparam as diferentes alternativas para gerar embeddings contextuais



BERT

- Na prática, BERT com *fine-tuning* atingiu o **estado da arte** em várias tarefas
- Usado o benchmark **GLUE**
 - *General Language Understanding Evaluation*



BERT

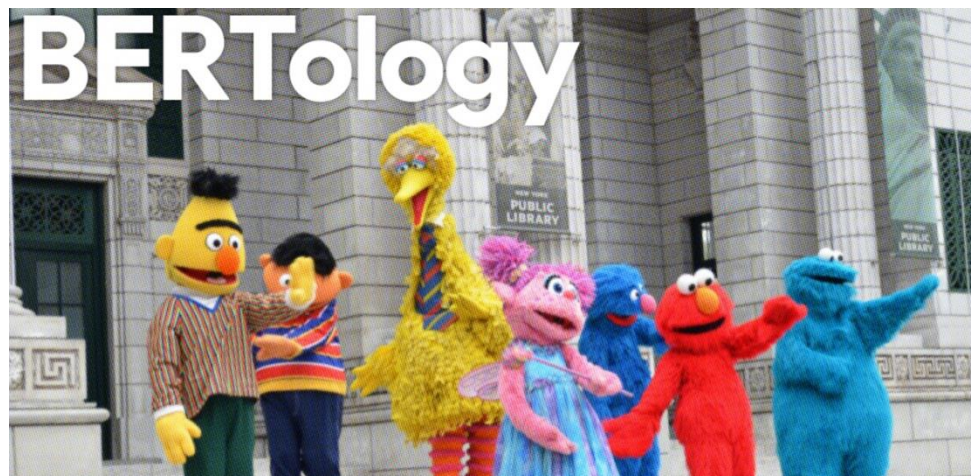
- GLUE: <https://gluebenchmark.com/>

GLUE Tasks

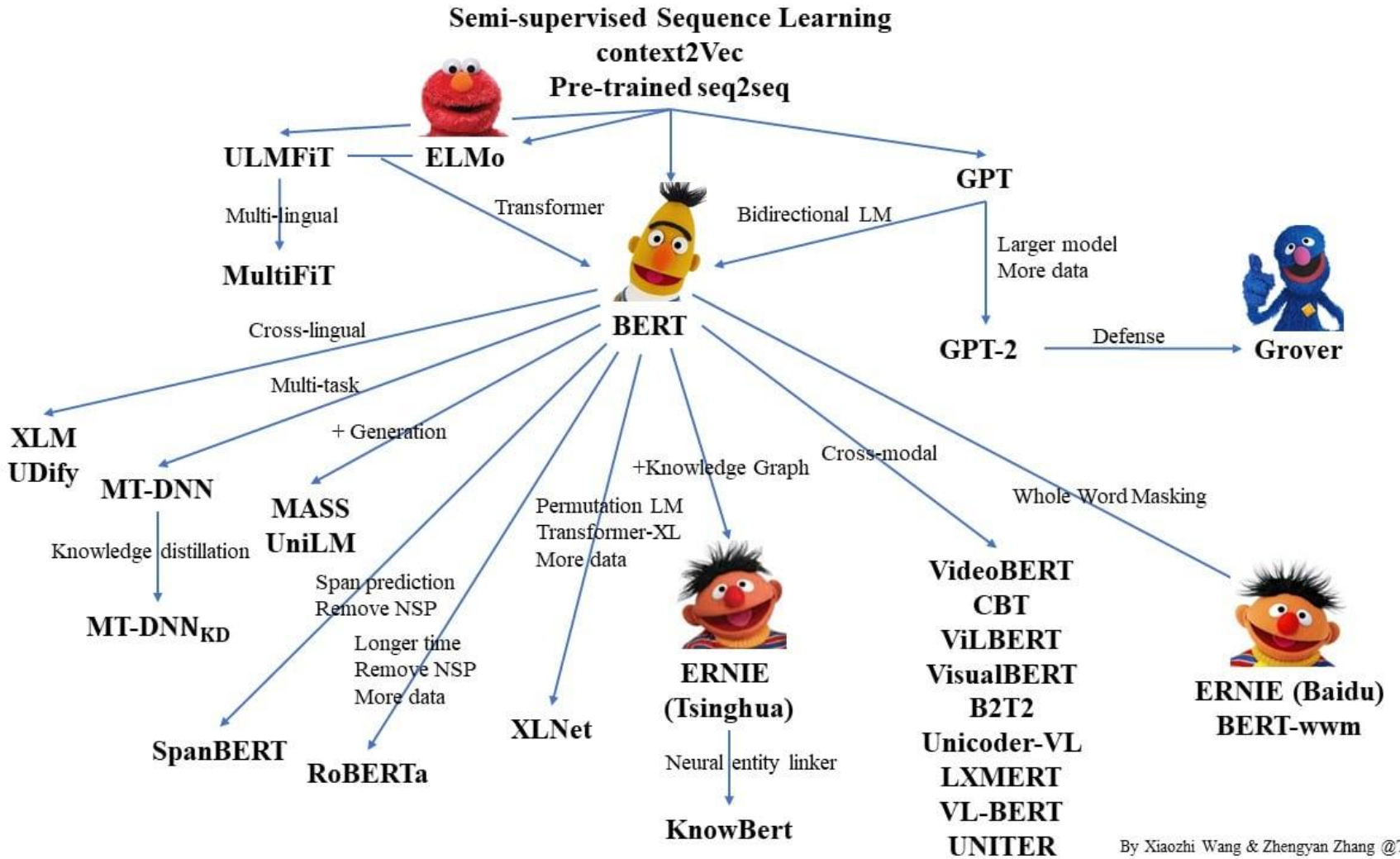
Corpus	Train	Test	Task	Metrics	Domain
Single-Sentence Tasks					
CoLA	8.5k	1k	acceptability	Matthews corr.	misc.
SST-2	67k	1.8k	sentiment	acc.	movie reviews
Similarity and Paraphrase Tasks					
MRPC	3.7k	1.7k	paraphrase	acc./F1	news
STS-B	7k	1.4k	sentence similarity	Pearson/Spearman corr.	misc.
QQP	364k	391k	paraphrase	acc./F1	social QA questions
Inference Tasks					
MNLI	393k	20k	NLI	matched acc./mismatched acc.	misc.
QNLI	105k	5.4k	QA/NLI	acc.	Wikipedia
RTE	2.5k	3k	NLI	acc.	news, Wikipedia
WNLI	634	146	coreference/NLI	acc.	fiction books

BERT

- *BERT-based models* se tornaram muito populares e foram amplamente propostos
- Sempre seguindo a ideia de melhores **representações contextuais e de sentença**



BERT



By Xiaozhi Wang & Zhengvan Zhang @THUNLP

Referências

- DEVLIN, Jacob et al. Bert: Pre-training of deep bidirectional transformers for language understanding. arXiv preprint arXiv:1810.04805, 2018.
- Mikolov, Tomas, et al. "Efficient estimation of word representations in vector space." arXiv preprint arXiv:1301.3781 (2013).
- Jay Alamar, The Illustrated BERT, ELMo, and co. (How NLP Cracked Transfer Learning): <https://jalammar.github.io/illustrated-bert/>
- WordPiece Tokenization: <https://huggingface.co/learn/nlp-course/chapter6/6>